Online Bus Monitoring & Management System

Final Year Project



Riphah International University

Submitted By

Moeez Raza	*63856*
Muneeba Munir	*66688*
Maham Maqsood	*66566*
Taqadus Noor	*62899*

FALL 2024 - SPRING 2026

Riphah College of Science & Technology

(RCST)

BMMS

Approval

Project Supervisor:

_____

_____

Name: _____

Date: _____ Signatures: _____

Project Manager:

_____

_____

Name: _____

Date: _____ Signatures: _____

Head of Department:

_____

_____

Name: _____

Date: _____ Signatures: _____

Dedication

To my department head at RCST, Dr. Tauseef Zahid, this project is offered with deep respect and appreciation. Sir Muhammad Ameer Hamza our project guide, respected teacher and a very special person, provided us with guidance and support while completing this project.

This endeavor is a tribute to every university student who travels on university buses and encounters discomfort due to unavailability of university buses, long wait times, inability to catch the bus and other problems associated with traveling to and from university, this project has been undertaken for the help of those students.

Executive Summary

A tool that shows where buses are at any given time helps students keep track of when they can catch rides - this makes planning daily moves easier. Instead of guessing, they see exactly where vehicles stand now, helping stay organized. When payments are overdue, reminders appear without needing extra steps. Backstage, school staff get their own dashboard where rules for different groups can be adjusted freely. From routing updates to collecting fees fairly, everything runs alongside each other smoothly.

An online bus monitoring and management system has been set up for the students to have a convenient access of knowing the actual position of their bus at a given point of time, enabling them to keep track of their schedules and payments. With an administrative portal for the administrations to manage all the buses from across the university, including implementing their own customized group policies to manage all buses more effectively. This will enable both administrators and the students to have a hassle-free approach to university bus administration.

Table of Contents

Approval i
Dedication ii
Executive Summary iii
Chapter 1: Introduction 2
1.1 Background 2
1.1.1 University Transportation System 2
1.1.2 Problems in Existing Bus Management 3
1.1.3 Need for Real-Time Bus Tracking 4
1.1.4 Need for Digital Bus Management 4
1.2. Problem Statement 5
1.3 Motivation 5
1.4 Goals and Objectives 6
1.4.1 Goal 6
1.4.2 Objectives 6
1.5 Project Scope 7
1.6 Literature Review 7
1.7 Existing Systems 7
1.8 Gap Analysis 8
1.8.1 Gap in the Traditional Transportation System 9
1.8.2 Gap in General-Purpose Navigation Applications 9
1.8.3 Gap in the Initial Arduino-Based Approach 9
1.8.4 Gap Related to Continuous GPS Tracking 10
1.8.5 Identified Gap and Proposed Direction 10
1.9 Proposed Solution 10
1.10 Project Methodology 11
Chapter 2: Software Requirements Specification 13
2.1 Introduction 13
2.2 Purpose 13
2.3 Intended Audience 14

2.3.1 System Administrators 14
2.3.2 Students 14
2.3.3 Drivers 14
2.3.4 Accountants 14
2.3.5 Bus Managers 14
2.4 Product Scope 14
2.5 Overall Description 15
2.5.1 Product Perspective 15
2.5.2 Product Functions 16
2.5.3 User Classes 16
2.5.4 Operating Environment 17
2.5.5 Constraints 17
2.5.6 Assumptions and Dependencies 18
2.6 System Actors 19
2.6.1 Administrator 19
2.6.2 Student 19
2.6.3 Driver 19
2.6.4 Accountant 19
2.6.5 Bus Manager 19
2.7 Functional Requirements 19
2.7.1 Authentication & Authorization 19
2.7.2 User Management 20
2.7.3 Bus Management 20
2.7.4 Stop Management 20
2.7.5 Route Management 21
2.7.6 Package Management 21
2.7.7 Real-Time Bus Tracking 22
2.7.8 Location Sharing 22
2.7.9 Administrative Dashboard 22
2.8 External Interface Requirements 23
2.8.1 Web Interface 23

2.8.2 Mobile Interface 23
2.8.3 REST API 23
2.8.4 WebSocket Interface 24
2.8.5 Map Interface 24
2.8.6 Database Interface 24
2.9 Non-Functional Requirements 25
2.9.1 Performance 25
2.9.2 Security 25
2.9.3 Reliability 25
2.9.4 Scalability 26
2.9.5 Usability 26
2.9.6 Maintainability 26
2.10 System Constraints 26
Chapter 3: System Analysis 29
3.1 Stakeholder Analysis 29
3.1.1 Students 29
3.1.2 Drivers 29
3.1.3 System Administrator 29
3.1.4 Accountant 30
3.1.5 Bus Manager 30
3.1.6 Parents 30
3.2 System Actors 30
3.3 Use Case Diagram 31
3.4 Use Case Descriptions 32
3.4.1 User Login 32
3.4.2 User and Group Management 32
3.4.3 Bus Management 32
3.4.4 Stop Management 32
3.4.5 Route Management 33
3.4.6 Package Management 33
3.4.7 Real-Time Bus Tracking 33

3.4.8 Location Access 33
3.4.9 History Management 33
3.5 System Workflow 33
3.6 Real-Time Tracking Workflow 34
3.7 Package Access Workflow 36
Chapter 4: System Design 39
4.1 System Architecture 39
4.2 Application Architecture 40
4.3 Database Design 42
4.3.1 ER Diagram 43
4.5 Class Diagram 44
4.6 Sequence Diagrams 45
4.7 Activity Diagrams 48
4.8 State Transition Diagram 52
4.9 Component Diagram 55
4.10 Deployment Diagram 56
4.11 Data Flow Diagram 57
4.12 API Architecture 59
Chapter 5: Implementation 64
5.1 Development Environment 64
5.2 Frontend Implementation 65
5.2.1 Login 65
5.2.2 Dashboard 65
5.2.3 Users 65
5.2.4 Groups 65
5.2.5 Buses 65
5.2.6 Stops 65
5.2.7 Routes 65
5.2.8 Packages 66
5.2.9 History 66
5.2.10 Locations 66

5.3 Backend Implementation 66
5.4 Database Implementation 67
5.5 Mobile Application Implementation 67
5.6 Authentication and Authorization 68
5.7 Real-Time Tracking Implementation 69
5.7.1 GPS Location Acquisition 69
5.7.2 WebSocket Connection 69
5.7.3 Location Transmission 70
5.7.4 Real-Time Broadcasting 70
5.7.5 Disconnection Handling 70
5.8 Package Management Implementation 71
5.9 Maps Integration 71
5.10 Security Implementation 71
5.10.1 JWT Authentication 72
5.10.2 Password Hashing 72
5.10.3 Group-Based Authorization 72
5.10.4 Read and Write Permissions 72
5.10.5 Protected APIs 72
5.10.6 CORS 72
5.10.7 Token Expiry 72
5.10.8 Application Settings 72
5.10.9 Input Validation 72
5.10.10 SQLAlchemy Database Protection 73
5.10.11 HTTPS 73
5.11 Version Control 73
5.12 Deployment 73
Chapter 6: Testing and Evaluation 75
6.1 Testing Strategy 75
6.2 Functional Testing 75
6.3 API Testing 75
6.4 Integration Testing 76

6.5 Mobile Application Testing 76
6.6 Real-Time Communication Testing 77
6.7 Performance Testing 77
6.8 Security Testing 77
6.9 Usability Evaluation 78
6.10 Testing Results 78
6.11 System Limitations 78
Chapter 7: Conclusion & Future Enhancements 80
7.1 Project Summary 80
7.2 Achievement of Objectives 80
7.3 System Benefits 81
7.4 Limitations 81
7.5 Lessons Learned 82
7.6 Future Enhancements 83
References i
Appendix A: Project Presentation ii

Chapter 1
Introduction

Chapter 1: Introduction

1.1 Background

University transportation plays a vital role in making sure all the students, faculty members, and staff reach their respective destinations with ease. In various educational institutions, transportation consists of several buses, drivers, their respective routes, stops, passengers and payments, etc. With the large volume of data increasing day by day, management through manual methods may not seem feasible anymore. Traditional transport management methods may work to some extent, but they fail to centralize all the communication between the passengers and the transport authorities efficiently.

The Online Bus Monitoring and Management System (OBMMS) aims to resolve the transportation needs of the university by having bus monitoring, route management, user management, payment and package record, and announcements as its main features. While the software was primarily designed keeping in mind the transportation needs of the university, it could be implemented in various organizations such as schools and colleges, etc., depending on their specific needs. There are various user types in the system including the Admin, Student, Driver, Bus Management, and Accountant. OBMMS aims to empower all the users by giving the former three the ability to keep track of the information digitally while providing the students with the feature to get all the bus-related information at the touches of a button. Drivers installed with a GPS system will enable the system to fetch their current location either through a mobile device or an android panel and update the users.

1.1.1 University Transportation System

A university's transport system comprises a fleet of buses, drivers, routes, stops, and passengers. Students, faculty members, and staff may rely on these services for routine transportation to and from the university. Within such a system, buses follow predetermined routes comprising multiple stops, while drivers are responsible for executing their own routes and transporting passengers. In the proposed system, transportation is organized with respect to customizable stops and routes. Stops can be created according to the requirements of the organization, and routes are designed by combining the required stops. A route can then be assigned to a bus, providing a structured relationship between buses, routes, and stops. The system also provides flexibility in bus and driver management. A driver can be associated with more than one bus where required, while each bus can be managed through the system's bus-related records. Unlike a system where students are restricted to a single bus, OBMMS allows students to view available information about transportation and to obtain the location of buses associated with the transportation service, subject to their active transportation package. This provides students with greater visibility of the available transportation service.

The system also incorporates transportation packages and payment records. Students can obtain a transportation package through the accountant, followed by a record of their payment and the activation of the package. The system maintains a history of these transactions and allows a package to be deactivated when required. Consequently, transportation management is not limited to monitoring buses but also incorporates administrative and financial aspects of the service.

1.1.2 Problems in Existing Bus Management

Traditional university bus management may involve manual recording of transportation details, route stops, student access, payment records, schedule updates, and other information. Such control can become difficult with multiple buses, drivers, routes, stops, and students. The project includes a survey to identify transportation problems faced by students and the necessity of a digital transportation management system. The survey consisted of 30 responses related to waiting time, missed buses, dues or payments issues, safety, route navigation, personnel, schedule changes, and mobile application interest. Survey results demonstrated the common problems in conventional transportation systems.

Among the identified issues, waiting time, skipped buses, and dues or payments problems were the most prevalent and repetitive concerns reported by the respondents. The majority also reported transportation schedule change communication issues and disruptions. In multiple responses, the transportation schedule change communication was rated as poor or bad, meaning that users are not consistently informed of transportation changes. The survey also aimed to determine the effect of bus delays on respondents. The majority of students identified such incidents as affecting their daily routines differently. There were responses indicating slight, moderate, and strong negative impacts of transportation delays on the users' schedules. These results imply that transportation issues can affect the lives of university students. Moreover, missed or skipped buses may add to the complexity of a person's schedule when using a conventional bus system. Next, the survey also aimed to learn about the convenience of the conventional bus route navigation. While some respondents have rated it as convenient to navigate, others have rated it as hard to understand.

The problem was also found in the necessity to rely on recorded or written manual instructions instead of a digital system providing transportation details online. Another issue was reported concerning the transportation payment and dues. Some students found it difficult to pay for the transportation tickets or not available at a convenient location. In a conventional system, such information may only be available online or in physical records with the responsible person in charge instead of a unified dashboard. The necessity of a digital transportation management system was identified through the survey as it may combine bus monitoring, record keeping, payment details, and other relevant instructions in one source.

1.1.3 Need for Real-Time Bus Tracking

One of the primary problems in university transportation is the uncertainty of passengers while waiting for the bus. A passenger without knowledge of the current location of a bus may experience confusion regarding the location of the bus, whether it is delayed, or whether it has passed their stop. This may cause unnecessary waiting and increase the possibility of missing the bus. Real-time bus tracking solves the problem by giving the passenger knowledge of the current geographical location of the bus through the digital map. In OBMMS, the driver's mobile device or Android panel provides GPS-based location information to the system. The location may then be shared with authorized users through the application. The student side of the system provides transportation-related information such as the current bus location, routes, stops, and other related bus information allowing students to track the movement of the buses rather than relying on manually communicated information.

Monitoring in real-time also brings benefits for transportation management. Administrators may gain more visibility of the buses operating in the transportation network, while route and bus information may be managed through a centralized system. The combination of GPS-based monitoring and customizable routes provide a basis for a more organized transportation operation. The survey conducted for this project also supports the need for improved transportation visibility. Waiting time and skipped buses were among the recurring issues reported by respondents, while many respondents expressed interest in a mobile application capable of solving the transportation problems identified in the survey. Therefore, real-time bus tracking is an essential component of the proposed system.

1.1.4 Need for Digital Bus Management

Real-time tracking is insufficient to satisfy the needs of a university transportation system. The full scope of a transportation solution requires management of users, buses, drivers, routes, stops, packages, payments, records, and communication. When these activities are handled separately or manually, the maintenance of accurate and up-to-date information becomes more difficult as the transportation system grows. OBMMS provides a centralized digital environment for managing these transportation components. The system includes modules for users, groups, buses, locations, stops, routes, packages, and history. Different access roles are provided according to the responsibilities of the users. These roles include, Admin, Student, Driver, Bus Management, and Accountant, allowing the system to separate transportation operations according to the user's responsibilities.

The route management component allows administrators to create and customize stops and use these stops to construct routes. These routes can subsequently be associated with buses, providing a structured representation of the transportation network. The location component supports GPS-based bus monitoring, while the history component maintains relevant records generated by system activities. The system also addresses transportation payment management.

The accountant can record a student's payment, after which the relevant transportation package becomes active. The system maintains the payment/package history and provides the ability to deactivate a package when required. This reduces dependence on disconnected manual records and provides a centralized mechanism for maintaining transportation-related financial information. Furthermore, the notification functionality provides a communication mechanism through which relevant transportation alerts can be delivered to users. This is particularly important when transportation information changes or when users need to be informed about relevant events.

Therefore, the proposed digital management approach combines real-time monitoring and administrative management within one platform. By incorporating transportation operations, user access, route configuration, bus monitoring, payment/package management, historical records, and notifications, OBMMS is intended to provide a more organized and digitally managed transportation environment for universities and other organizations with similar transportation requirements.

1.2. Problem Statement

Students face a lot of problems while traveling by bus regarding the arrival of the bus at the stopping point, whether it has arrived or not, causing them to reach late, a long wait for the bus, and missing the bus. Students also have questions about their fee dues and whether their transport package is active or not, as they cannot travel by bus if their package is inactive. Therefore, there should be some sort of system that provides the students with live location of the bus without compromising the privacy of others, hence only those who are provided permission or have an active package should be able to track their bus in real-time. Moreover, a full management system that sets the permissions according to individual designations should be there. This can be applicable for a university as well as any company providing bus/van services, and it can be adjusted according to the company's needs.

1.3 Motivation

The motivation for developing the Online Bus Monitoring and Management System (OBMMS) came from the transportation-related problems personally experienced by the project team in the university environment. Students often rely on the university transportation for their daily travel, but their confusion about bus arrival and waiting times can create inconvenience and impact their daily routines. The lack of a method for students to monitor the real-time location of a bus can heighten the uncertainty.

The project team also conducted a survey with 30 respondents to investigate the problems associated with the conventional university transportation system. The responses revealed recurring issues like waiting time, skipped buses, payment or dues-related problems, difficult access to transportation information, and inadequate communication of schedule changes. These experiences motivated the project team to develop a digital solution to address the transportation problems.

Another major motivation came from the usage of MS Word and MS Excel to maintain transportation-related information. While they can be useful for record storage, managing transportation information in separate documents does not give students a centralized, real-time platform. Schedule information was not consistently available to students. This motivated moving from fragmented manual and document-based management to a centralized digital transportation system. Thus, the project team aimed to introduce a real-time bus tracking so that students could view the current location of buses instead of relying on manually conveyed information. With the GPS information acquired from the driver's mobile or Android panel, OBMMS gives students access to live bus location information along with the relevant bus schedule. In addition to solving the bus tracking problem, the team wanted to bring together different transportation management activities in a single platform. Instead of managing buses, routes, stops, users, and payment records in different processes, OBMMS brings these functions together in a system. The payment management component also enables activating transportation packages after payment is recorded while maintaining a digital payment history.

Overall, the motivation for OBMMS came from developing a practical and centralized transportation solution that responds to problems seen in the existing university transportation system, using the findings in the project's survey, reducing manual Word and Excel-based processes, providing real-time bus visibility and bringing transportation and payment management into a unified digital platform.

1.4 Goals and Objectives

1.4.1 Goal

The main purpose of the Online Bus Monitoring and Management System is to design a central online system used to monitor and manage university buses. It is expected that the proposed solution will help optimize transportation services by providing real-time bus location tracking, bus schedules, digital transportation package and payment management, and centralized administration of transportation means, routes, stops, and users.

1.4.2 Objectives

The main objectives of the proposed system are as follows:

- To enable real-time tracking of university buses so that students can view the current location of buses from within the system.
- To reduce student waiting time and the risk of missing buses by providing students with an accessible way to view bus schedules and real-time location.
- To enhance transportation information accessibility by providing students with a centralized platform to view bus schedules, routes, stops, and bus location in real-time.
- To provide a secure digital fee and package management mechanism that reduces reliance on manual paperwork and facilitates the accurate maintenance of payment records.

- To digitally manage transportation packages by activating a student's package after recording the payment and maintaining their corresponding payment history.
- To centralize the management of users, buses, drivers, routes, and stops through role-based access, for administrators and other authorized personnel.
- To enable customizable transportation routes and stops according to the requirements of the university or other organizations using the system.
- To maintain digital transportation and payment records so that the relevant historical information can be accessed when required.
- To provide payment-related notifications to improve communication between the transportation management system and the students about their payment or package status.
- To reduce the reliance on separate MS Word and MS Excel files, by integrating major transportation management functions into a centralized digital system.
- To provide a scalable transportation management approach that primarily serves universities, but can also be adapted to schools, colleges, and other organizations, according to their transportation requirements.

1.5 Project Scope

The scope of the Online Bus Monitoring and Management System (OBMMS) includes the digital monitoring and management of the transportation services for the universities and other organizations that have a bus transportation facility. While the system will mainly focus on the university's needs, the configurable route, stops, buses, users, transportation packages, and other management functions can align according to the different educational facilities' and organizations' requirements. The system will provide a centralized place to manage transportation-related information rather than using multiple MS Word and MS Excel documents. The scope of the system includes the management of users, buses, routes, stops, locations, transportation packages, payment records, and historical data.

1.6 Literature Review

The development of real-time transportation management systems has attracted much interest in the field of Intelligent Transportation Systems (ITS). Previous researchers have investigated the use of Global Positioning System (GPS), Automatic Vehicle Location (AVL), mobile applications, and centralized data-management platforms to enhance the monitoring and management of buses. The literature reveals that real-time transportation information can benefit both passengers and transportation administrators by providing more accessible information about vehicles' movement and enhancing the availability of operational data.

1.7 Existing Systems

There are several digital applications and transportation technologies that are available for vehicle navigation and location-based services. Popular applications like Google Maps offer users mapping, navigation, and location-related services. These applications are designed to

provide general-purpose transportation and navigation services but are not specifically developed to manage the complete transportation operations of a university or an organization. Google Maps can be useful in visualizing geographical locations and receiving navigation information; however, it does not offer the specific access control and transportation management features required by the proposed Online Bus Monitoring and Management System (OBMMS). The requirements of a university transportation system might involve providing access control to different users, managing buses and drivers, customizable stops and routes, package management, fee records, and transportation-related history. In the environment targeted by the OBMMS, there was no dedicated digital system available previously for managing the complete university transportation process.

The transportation-related information and records were managed using manual methods like MS Word and MS Excel. Students relied on the manual communication of transportation information and could not access the live location of their university buses. Similarly, the payment and dues information were maintained manually. The lack of a dedicated transportation management system created a requirement for a platform that could offer transportation monitoring and administrative management in a single system. OBMMS was developed to offer a centralized platform with role-based access for Admin, Student, Driver, Bus Management, and Accountant users. Unlike a general-purpose public navigation application, OBMMS is developed keeping in mind the requirements of an organization.

The system offers live bus-location information, bus and driver management, customizable stops and routes, package management, recording of payment, and payment history with access control features. Once the accountant records the payment of a student, the respective package can be activated and the payment history is maintained digitally. Thus, from the existing-system analysis, it can be stated that general-purpose applications can offer location and navigation services but the specific combination of real-time university bus monitoring, role-based access control, route and bus management, and fee/package management was not available in the previous manual setup. OBMMS offers these requirements through a dedicated and configurable transportation management platform.

1.8 Gap Analysis

The analysis of existing transportation practices and the initial design approach identified several gaps that influenced the development of the Online Bus Monitoring and Management System (OBMMS). The transportation process considered for the project was primarily dependent on manual methods, including MS Word, MS Excel, and communication with university staff. In addition, general-purpose applications such as Google Maps provide navigation and location services but do not provide the access control, transportation administration, and fee-management functions required by an organization.

1.8.1 Gap in the Traditional Transportation System

The previous manual process did not provide students with direct access to the current location of their university buses. The students had to rely on manually provided schedules and transportation details. Additionally, transportation features such as bus, route, stop, and payments had to be managed manually.

The following are the gaps that were identified in the traditional transportation system:

- There was no centralized system for monitoring university buses
- There was limited direct access to current bus location and transportation information
- There was a need to manage transportation payments and dues manually
- There was limited centralized management of buses, drivers, routes, and stops
- There was no role-based access control system for transportation users
- There was a challenge in consolidating transportation information in one platform

OBMMS addresses these gaps by providing a centralized digital platform for transportation monitoring and management.

1.8.2 Gap in General-Purpose Navigation Applications

Public applications like Google Maps provide geographical maps and navigation services but are not designed to specifically manage an organization's transportation operations. They do not provide the project specific combination role-based access, bus management and route/stop configuration, package management and fee records required by OBMMS. Therefore, the gap was not simply one that there was no mapping service. Requirement was the organization specific transportation management system where authorized users could access information based on their roles and administrators could manage transportation related data from a centralized platform.

1.8.3 Gap in the Initial Arduino-Based Approach

During the initial planning stage, the project team considered developing the bus tracking component using an Arduino-based hardware tracker. Although a hardware-based solution would allow GPS-based tracking, the team identified challenges related to cost, maintenance, accessibility, and scalability. An Arduino-based tracker would require dedicated hardware to be installed and maintained on buses. This could complicate the overall implementation and maintenance requirements, especially if multiple buses are to be monitored. In addition, students would still have to use a separate application or interface to access the collected location information. These factors created a gap in the initial hardware-based tracking approach and the project's goal of providing an accessible and scalable transportation management solution.

1.8.4 Gap Related to Continuous GPS Tracking

The group of people also evaluated the continuous GPS tracking through the mobile application. However, constant tracking can have unwelcome impacts such as extra battery drainage and privacy problems if the data about the location is shared. To avoid the issues, the implemented approach is designed to share the bus location when the OBMMS application is installed and running on the Android device panel. The approach would provide the needed data about the location while avoiding the need for unrestricted continuous location sharing when the tracking application is not running.

1.8.5 Identified Gap and Proposed Direction

The analysis identified the need to develop a solution that will address such aspects:

- Bus location tracking in real-time;
- Availability in most common mobile devices;
- Reduced complexity of hardware components;
- Role-based access control;
- Transportation, bus, driver, and stop management;
- Management of digital packages and fees;
- Payment history tracking;
- Transportation management configuration for different organizations;
- Optimization of computing resource usage in line with the tracking needs.

With these gaps in mind, the project was redirected from developing the Arduino-based tracking prototype to creating a mobile-application-based tracking solution. In this version, Android devices serve as sources of information about the location of buses, while the application provides access to this information for authorized users. In this way, the identification of gaps in the initial prototype of the transportation management system helped redirect the development efforts to prototyping the mobile application for tracking buses. The new solution was designed to address the complex of issues associated with the creation of solutions based on hardware components. The transition to the application-based concept has allowed for reducing the complexity and cost of the system while providing a better user experience due to the utilization of familiar interfaces. In addition, the new concept covers the management of transportation-related objects, which were unavailable in the initial solution.

1.9 Proposed Solution

To address the limitations identified in the existing transportation process and the initial hardware-based tracking approach, the **Online Bus Monitoring and Management System (OBMMS)** was proposed as a centralized, mobile-enabled transportation management solution. The system is designed to provide real-time bus-location information while also supporting the administrative and financial requirements of organizational transportation.

The proposed system replaces the dependence on manual transportation records maintained through MS Word and MS Excel with a centralized digital platform. It provides different levels of access according to user roles, allowing each type of user to interact with the system according to their responsibilities. The main roles supported by OBMMS are **Admin, Student, Driver, Bus Management, and Accountant**.

1.10 Project Methodology

The Online Bus Monitoring and Management System (OBMMS) was developed using an **iterative and structured software development methodology**. The methodology was selected to allow the project team to progressively analyze requirements, design the system, implement individual components, identify technical issues, and refine the system through debugging and further planning.

The development process consisted of the following major phases:

Requirement Gathering → Analysis → Planning → UI/UX Design → Implementation → Debugging → Deployment → Final Testing → Presentation

The project's documented development process also included a feedback loop from the debugging phase back to planning. This allowed identified problems and implementation issues to be considered in subsequent development activities.

Chapter 2
Software Requirements Specification

Chapter 2: Software Requirements Specification

2.1 Introduction

The Online Bus Monitoring and Management System (OBMMS) is a web and mobile based system that was created in order to manage university transportation services and provide real-time bus monitoring. The system provides a centralized platform through which authorized users can manage users, groups, buses, stops, routes, locations, packages, and related history. OBMMS consists of an administrative web dashboard, a mobile application for clients, and a backend server that is responsible for authentication, authorization, data management, and real time communication. The system uses a role- and permission-based access control mechanism, allowing administrators to define groups and assign specific permissions to users. The system also provides real-time bus location tracking. A driver can log into the system using an Android device, such as a mobile phone or an Android panel that is installed inside a bus. The device obtains the bus location and communicates it to the backend server. Authorized students can then receive and view the bus location through the client application. The system is designed to provide a flexible and centralized solution for managing university transportation operations while reducing the need for manual management of buses, users, routes, stops, and transportation packages.

2.2Purpose

OBMMS is created to be a centralized system for managing and monitoring university buses. The system is designed to offer several purposes, which include:

- Manage users and their access permissions.
- Provide customizable user groups.
- Manage buses and their associated information.
- Manage transportation stops.
- Manage routes and their schedules.
- Manage user transportation packages.
- Maintain package and payment-related history.
- Provide real-time bus location tracking.
- Allow authorized users to access transportation information.
- Provide administrators with a centralized dashboard for managing the system.
- Provide controlled access to system resources through permissions.

The system also aims to provide a low-cost tracking solution by allowing any suitable Android device to act as the bus tracking device instead of requiring dedicated GPS hardware.

2.3 Intended Audience

The intended audience of OBMMS consists of the following users:

2.3.1 System Administrators

Administrators are responsible for managing the overall system. They can manage users, groups, buses, stops, routes, locations, packages, and history according to their assigned permissions.

2.3.2 Students

Students are clients of the system who can use the mobile application to access transportation-related information and monitor the location of buses for which they have access.

2.3.3 Drivers

Drivers use an Android device to log into the system and provide the real-time location of the bus they are operating.

2.3.4 Accountants

Accountants are users whose access can be configured according to their responsibilities. They can be given appropriate permissions for managing or viewing package and financial-related information.

2.3.5 Bus Managers

Bus managers are users who can be assigned permissions related to transportation management including buses, stops, routes and other resources. The system does not permanently restrict these roles and user groups are customizable, permissions can be assigned according to the requirements of the organization.

2.4 Product Scope

OBMMS covers the entire management and monitoring process of university's transportation services through a centralized software platform. The system comprises functionalities of user management, group and permissions management, bus management, stop management, route management, location management, package management and history management.

The system provides two major business interfaces for users which are:

1. Administrative Web Dashboard
2. Client Mobile Application

The administrative dashboard includes functionalities of Create-Read-Update-Delete (CRUD) for the major resources of the system. The access to these sets of operations is achieved through customized permissions. The mobile application is a client application for students and

does not comprise the CRUD operations. The application is aimed to provide transportation-related information and real-time bus monitoring functionalities to authorized users. In addition, the system leverages on Android devices for location tracking in real-time. The driver can access the system from an android phone or an android panel that is installed in a bus. In this case, the phone or the panel will send its location records to the backend and the information will be available to the designated students who are authorized to access this data. With respect to mappings, OBMMS utilizes OpenStreetMap. In this regard, the web application involves React-Leaflet while the mobile application uses WebView-based approach. In short, the system does not use Google Maps.

2.5 Overall Description

2.5.1 Product Perspective

OBMMS is a centralized client-server system comprising several software components. The system can be separated into three primary layers:

Client Layer → Backend Layer → Database Layer

The client layer comprises the administrative web application and the mobile application. The backend layer provides REST APIs, authentication, authorization, CRUD, and real-time communication. The database layer stores the system's information, including users, groups, buses, stops, routes, packages, locations, and history.

For tracking the location of buses in real-time:

Driver Android Device → Backend Server → (Authorized Student Clients)

The driver's Android device acquires the current location and communicates with the backend. The backend processes the location and provides it to connected and authorized clients. The system, therefore, combines conventional request-response communication via REST APIs and bus location tracking using real-time communication. OBMMS is a centralized client-server system comprising several software components.

The system can be separated into three primary layers:

Client Layer → Backend Layer → Database Layer

The client layer comprises the administrative web application and the mobile application. The backend layer provides REST APIs, authentication, authorization, CRUD, and real-time communication. The database layer stores the system's information, including users, groups, buses, stops, routes, packages, locations, and history.

For tracking the location of buses in real-time:

Driver Android Device → Backend Server → (Authorized Student Clients)

The driver's Android device acquires the current location and communicates with the backend. The backend processes the location and provides it to connected and authorized clients. The system, therefore, combines conventional request-response communication via REST APIs and bus location tracking using real-time communication.

2.5.2 Product Functions

The major functions of OBMMS include:

- User authentication.
- JWT-based session management.
- Customizable user groups.
- Permission-based access control.
- User management.
- Bus management.
- Stop management.
- Route management.
- Location management.
- Package management.
- History management.
- Real-time bus tracking.
- Location sharing between the tracking device, backend, and authorized clients.
- Administrative dashboard.
- Mobile client application.
- Map-based bus visualization.

The administrative dashboard provides CRUD operations for the major resources. The system allows permissions to be configured as not allowed, read-only, or read-and-write according to the requirements of each user group.

2.5.3 User Classes

OBMMS provides a customizable user-group system rather than relying on fixed roles only.

The currently defined user groups include:

- **Admin**
- **Student**
- **Driver**
- **Accountant**
- **Bus Manager**

The Admin group is available by default. However, groups and their permissions can be customized according to organizational requirements.

Permissions can be assigned at different levels, including:

- **Not Allowed** the user cannot access the resource.
- **Read Only** the user can view the resource but cannot modify it.
- **Read and Write** the user can view and modify the resource.

This permission mechanism allows the same system to be adapted to different organizational requirements without changing the underlying application.

2.5.4 Operating Environment

OBMMS operates in a web, server, database, and Android environment.

Web Environment

The administrative interface is accessed through a modern web browser. The frontend is developed using React-based technologies and provides the administrative dashboard and map-based interfaces.

Backend Environment

The backend is developed using FastAPI and provides REST APIs and real-time communication services.

Database Environment

The system uses **MySQL** as the relational database management system, with **SQLAlchemy** used for database interaction.

Mobile Environment

The client application operates on Android devices. An Android device can also be used as a bus tracking device when a driver logs into the system.

The tracking device can be either:

- An Android mobile phone.
- An Android panel installed inside the bus.

Mapping Environment

The system uses **OpenStreetMap** for geographical map data. **React-Leaflet** is used for map visualization in the web application, while the mobile application uses a WebView-based approach.

2.5.5 Constraints

The major constraints of OBMMS include:

1. The system requires an Internet connection to communicate with the client, tracking device, and the backend server.
2. Real-time bus tracking relies on the availability of GPS/location services on the Android tracking device.
3. The tracking device should have an Android environment where the client application can run.
4. The accuracy of the displayed bus location relies on the accuracy of the location information provided by the Android device.
5. The system uses OpenStreetMap and not commercial map services such as Google Maps.
6. The mobile application is for client-side access only and does not provide administrative CRUD features that are available in the web dashboard.
7. System resources are accessed using configured permissions; therefore, users cannot access resources that they are not permitted to use.

2.5.6 Assumptions and Dependencies

Assumptions

The system is designed using the following assumptions:

- Users are provided with valid system credentials.
- Drivers use an Android device capable of obtaining location information.
- Students use a supported mobile device to access the client application.
- The backend server is available when real-time tracking or server-based operations are required.
- Users are assigned appropriate groups and permissions by authorized administrators.
- Bus and route information entered into the system is maintained by authorized users.
- Location services on the tracking device are enabled when real-time tracking is required.

Dependencies

OBMMS depends on:

- FastAPI backend services.
- MySQL database.
- SQLAlchemy database interaction layer.
- Android location services.
- Internet connectivity.
- OpenStreetMap.
- React-Leaflet for web map visualization.
- WebView-based map/client functionality in the mobile application.
- JWT-based authentication mechanism.

2.6 System Actors

The primary actors of OBMMS are:

2.6.1 Administrator

The administrator manages the overall system and can manage users, groups, buses, stops, routes, locations, packages, and history according to the permissions assigned to the administrator. The administrator also controls the permission structure of other user groups.

2.6.2 Student

The student uses the mobile application as a client to access transportation-related information and monitor available buses.

2.6.3 Driver

The driver logs into the system through an Android device. The device obtains the current location and sends it to the backend for real-time bus monitoring.

2.6.4 Accountant

The accountant is an organizational user who can be assigned permissions related to package and financial information.

2.6.5 Bus Manager

The bus manager can be assigned permissions for transportation-related resources such as buses, stops, routes, and locations. Since the permission system is customizable, the exact capabilities of each actor depend on the permissions assigned to their group.

2.7 Functional Requirements

2.7.1 Authentication & Authorization

The system shall provide username/password authentication. The system shall make use of JSON web tokens to manage authenticated sessions.

The system shall:

- Allow users to log in using a username and password.
- Generate a JWT after successful authentication.
- Verify the JWT when protected resources are accessed.
- Refresh the token when required during verification.
- Allow the token expiry period to be configured through system settings.
- Use 30 days as the default token expiry period.
- Prevent unauthorized users from accessing protected resources.

The authorization system shall evaluate the permissions assigned to the user's group before allowing access to system resources.

2.7.2 User Management

The system shall provide CRUD operations for users. Authorized users shall be able to:

- Create users
- View users
- Update user information
- Delete users
- Assign users to groups
- Control access to user-related resources via permissions

The system shall not require pre-defined roles for all users. User groups can be tailored to the needs of the organization.

2.7.3 Bus Management

The system shall provide CRUD operations for buses. Each bus shall contain information such as:

- Bus ID
- License
- Capacity
- Number of passengers
- Current location reference
- Assigned driver
- Active status

The system shall allow authorized users to perform create, view, update, and delete operations on bus records in accordance with their assigned permissions. The active status shall enable the system to determine if a bus is active within the management system.

2.7.4 Stop Management

The system shall support CRUD operations for transportation stops. Stop shall have:

- Stop ID
- Stop name
- Description
- Latitude
- Longitude
- Campus (true/false)
- Active (true/false)

Latitude and longitude are geographical coordinates, that allow to represent this stop on the map. The **is_campus** field indicates if this stop represents a campus. Users shall be able to add/edit/view/delete stops depending on their permissions.

2.7.5 Route Management

The system shall support the CREATE, READ, UPDATE and DELETE functions for routes. The route consists of the following information:

- Route ID
- Departure
- Destination
- Bus
- Scheduled time
- Active

The routes describe the transportation relationship between the departure and destination and a bus available for these stops. Authorized users can manage the records according to their permissions.

2.7.6 Package Management

System shall provide CRUD operations for transportation packages. Each package shall contain:

- Package ID
- User
- Price
- Month
- Year
- Active status

Packages shall relate to respective users and represent their transportation package for one particular month and year. The system shall allow the authorized users to:

- Create packages
- View packages
- Update packages
- Delete packages
- Activate/deactivate packages

The system shall also maintain the history of the packages containing:

- History ID
- Package
- User

- Amount
- Date
- Time

History packages shall represent some sort of transaction or financial interaction related to the primary packages stored by the system.

2.7.7 Real-Time Bus Tracking

The system shall provide real-time bus location tracking. A driving personnel can log into the system using an android device. The device can be:

- An android mobile phone.
- An android panel installed inside the bus.

The android device acquires the current geographical location of the bus and communicates it with the backend. The real-time tracking process follows the general flow below:

Driver → Android Tracking Device → Backend → Authorized Student Clients

The backend handles the received location information and makes it available to authorized clients through real-time communication. The system therefore does not require a dedicated hardware GPS tracker for each bus. An appropriate Android device can perform the tracking function.

2.7.8 Location Sharing

The system must enable sharing of the bus location information between the tracking device and authorized clients. The process of sharing location information is as follows:

1. The Driver logs in to the Android tracking device.
2. The device gets the bus's geographical location.
3. The location is sent to the backend.
4. The backend processes the location.
5. The location is shared with authorized student clients.
6. The students are able to view the bus location on the client application.

The location can be displayed on the map using the geographical coordinates.

2.7.9 Administrative Dashboard

The system shall provide a web based administrative dashboard. The dashboard shall provide management interfaces for the system's major resources including:

- User
- Groups
- Buses

- Stops
- Routes
- Locations
- Packages
- History

The dashboard shall provide create, read, update, delete (CRUD) capabilities for whichever resources the logged-in user has permission to manage. The permission system shall be designed such that administrators can give groups of users any of the following permissions on each resource:

- No access
- Read only
- Read and write

This allows the administrative dashboard to be used by a variety of different organizational users while still providing a level of security by eliminating the ability for some users to have complete access over all resources.

2.8 External Interface Requirements

2.8.1 Web Interface

The web interface will provide access to the administrative dashboard for authorized users. The interface will allow users with appropriate permissions to manage system resources using CRUD operations. The web interface will also provide map-based visualization using OpenStreetMap and React-Leaflet. The interface will restrict operations according to the permissions assigned to the authenticated user.

2.8.2 Mobile Interface

The mobile application shall have a client interface for students and any other authorized client. The mobile application shall provide transportation-related information and real-time bus monitoring. The mobile application shall not provide the administrative CRUD functionality available in the web dashboard. The mobile application shall use a WebView-based approach for the relevant web/map functionality.

2.8.3 REST API

The backend shall provide REST APIs for communication between the client applications and the server.

REST APIs shall be used for operations related to the system's CRUD resources, including:

- Users
- Groups

- Buses
- Stops
- Routes
- Locations
- Packages
- History

The API shall authenticate and authorize requests according to the user's JWT and assigned permissions. The API shall support the required operations for creating, reading, updating, and deleting resources.

2.8.4 WebSocket Interface

The system shall use WebSocket communication for real-time bus location functionality. Unlike conventional request-response communication, WebSocket communication allows the system to maintain a real-time communication channel for location updates. The WebSocket-based communication shall support the flow of location information from the driver's tracking device through the backend to authorized clients. The WebSocket interface is therefore primarily responsible for the real-time component of the OBMMS system.

2.8.5 Map Interface

The system shall provide geographical visualization using OpenStreetMap. The web application shall use React-Leaflet to display map information. The map interface shall be capable of displaying geographical locations using latitude and longitude coordinates. Bus and stop locations can therefore be represented visually instead of only displaying numerical coordinates. The system does not depend on Google Maps for its mapping functionality.

2.8.6 Database Interface

The system shall use MySQL as its relational database management system. SQLAlchemy shall be used as the database interaction layer between the FastAPI backend and MySQL database.

The database shall store system information including:

- Users
- Groups
- Permissions
- Buses
- Stops
- Routes
- Locations
- Packages
- History

The backend shall communicate with the database through the application's database layer rather than allowing clients to directly access the database.

2.9 Non-Functional Requirements

2.9.1 Performance

The system shall be responsive to communication between clients and the backend. The real-time location tracking component of the system shall enable frequent location updates to allow students to know the current bus position with a slight delay.

The target requirements for the system shall entail:

- Bus location update every 10 seconds on average
- Map/location information should be provided with a target latency of less than 3 seconds under normal network conditions

The actual performance may vary depending on the availability and quality of the network connection, GPS accuracy, server performance and client device.

2.9.2 Security

The system shall protect its resources from unauthorized use. The security requirements are given below:

- Username and password authentication.
- JWT-based authentication.
- Token verification for protected resources.
- Configurable token expiry.
- Password protection.
- Permission-based authorization.
- Protected API endpoints.
- CORS configuration.
- Restriction of CRUD operations according to user permissions.

The system shall ensure that users cannot perform operations for which their assigned group does not have the required permission.

2.9.3 Reliability

The system shall operate consistently under normal conditions. The system backend should be able to process the client's request and manage the database effectively. In addition, for the system to provide real-time tracking, there is a need to ensure reliable communication between the android tracking device, backend, and clients.

The real-time tracking function relies on the following services:

- Internet connectivity.
- GPS/location services.
- Android tracking device.
- Backend server.

If any of the services fail, the system will be unable to update the location in real-time until the services are restored.

2.9.4 Scalability

The system shall be designed in such a way that additional users, buses, stops, routes, and packages may be added without altering the fundamental system architecture. The use of a centralized FastAPI backend and MySQL database enables the system resources to be managed through a common infrastructure. The real-time WebSocket communication mechanism provides a foundation for supporting multiple clients receiving bus location updates. The actual maximum number of simultaneous users and connections depends on the deployed server's hardware, network capacity, database configuration, and WebSocket implementation.

2.9.5 Usability

The system will ensure there is an interface understandable and usable by different categories of users. The administrative dashboard will provide appropriate resources organized according to management operations, including users, groups, buses, stops, routes, packages, and history. The permission system will enhance usability by presenting relevant operations for a given user. The mobile application will ensure there is a simplified client interface focused on transportation information and bus monitoring.

2.9.6 Maintainability

The system shall be maintainable through separation of the major components. The project is divided into frontend, backend, and mobile components. The backend uses FastAPI and SQLAlchemy, whereas the database uses MySQL. This approach allows individual components to be modified and maintained without unnecessarily altering the complete system. In addition, the use of the CRUD-based resource management provides a consistent structure for maintaining the different system entities. Version control through the Git and GitHub shall be used to maintain the project's source code and development history.

2.10 System Constraints

The following are the constraints that will be used in the implemented OBMMS system:

1. **Internet Dependency:** Communication, including tracking location of the tracking device and communicating with clients requires internet connectivity.
2. **GPS Dependency:** The application relies on the GPS or location services of the Android tracking device for location tracking. The accuracy of GPS information may vary depending on the Android version or device.

3. **Android Dependency:** A user must have an Android device that is capable of installing the tracking application and provide GPS information.
4. **Server Dependency:** The application depends on the availability of the server for client applications to perform authentication, CRUD operations, and real-time tracking of the vehicle.
5. **MySQL Dependency:** The system relies on the availability of the MySQL database to store the system data.
6. **Permissions Dependency:** The application will only allow a user to perform actions depending on the permissions granted. Users will not have access to resources or perform actions that are not allowed.
7. **Map Dependency:** The mapping feature relies on OpenStreetMap information and availability of the required map resources.
8. **Mobile Application Limitation:** The mobile application is a client application and does not support all the features of the web dashboard, including the full CRUD operations of the dashboard.
9. **Tracking Accuracy Dependency:** The accuracy of the tracking information depends on the GPS or location information provided by the tracking device and network availability.
10. **Configurable Authentication Expiry:** The expiration time of the JWT authentication token can be configured in the application's configuration file. By default, the application is set to 30 days.

Chapter 3
System Analysis

Chapter 3: System Analysis

3.1 Stakeholder Analysis

The stakeholders are those who interact with the system, Online Bus Monitoring and Management System (OBMMS) use it services, manage the system data or are impacted by the system. The main stakeholders of the current system are the students, drivers, system administrators, and management staff such as accountants and bus managers. The current system has a permission-based access control mechanism; instead of having a fixed functionality assigned for each type of user, it allows the users to be grouped together, and particular permission to be assigned to the particular group. The university management can make the user profiles according to their requirements and assign the required groups to them. The main stakeholders are described below.

3.1.1 Students

Students are the main users of the mobile application. They can log in to the system and view transport related information according to the permissions provided and the active package. The students can view available buses, routes, stops and the current location of buses, if they have the necessary permissions. If the student does not have the required permission for location, an active package is required for the student to view the location of buses. Thus, there is a controlled mechanism for students to view real-time transportation information.

3.1.2 Drivers

The drivers are tasked with providing the current location of the bus. In this step, the driver logs into a mobile application, chooses the bus that he is operating, and shares the GPS location of the bus. The location is sent from the driver's Android device to the backend through a WebSocket connection. The backend then provides the real-time location to authorized students. A driver can be assigned to more than one bus. The driver selects the appropriate bus before starting location sharing.

3.1.3 System Administrator

The system administrator has the highest level of control over the system. The administrator can perform CRUD operations on the major system resources and manage users, groups, permissions, buses, stops, routes, packages, and other available system data. The administrator can also manage user groups and assign permissions to them. However, location sharing itself is not an administrator-only function. Location sharing is performed by users whose assigned group has the required location Read and Write permission. The administrator also has the authority to delete history records. No other group has permission to delete history.

3.1.4 Accountant

An accountant is a permission-based user of the system. The accountant does not have a permanently fixed set of functions. The permissions assigned to the accountant's group determine the operations that can be performed. For example, an accountant whose group has Package Read and Write permission can activate or deactivate packages. Package-related operations therefore depend on the permissions assigned to the user's group.

3.1.5 Bus Manager

The Bus Manager is also a permission-based user. The system does not force a fixed set of functions for this group. Instead, the required permissions can be assigned to the group according to the university's management requirements.

3.1.6 Parents

Parents can be considered potential future stakeholders of OBMMS. However, parent-specific functionality is not included in the current implemented version of the system. Therefore, parents are not considered active users of the current system.

3.2 System Actors

An actor represents a user or external entity that interacts with the system. In OBMMS, access is controlled through users and groups. A user is assigned to a group, and the permissions of that group determine the operations that the user can perform.

The major actors of the system are:

- Student
- Driver
- Administrator
- Accountant
- Bus Manager

The Accountant and Bus Manager do not have completely fixed functionality. Their available operations depend on the permissions assigned to their groups. Similarly, other university management staff can also use the system by creating a user profile and assigning the required group and permissions.

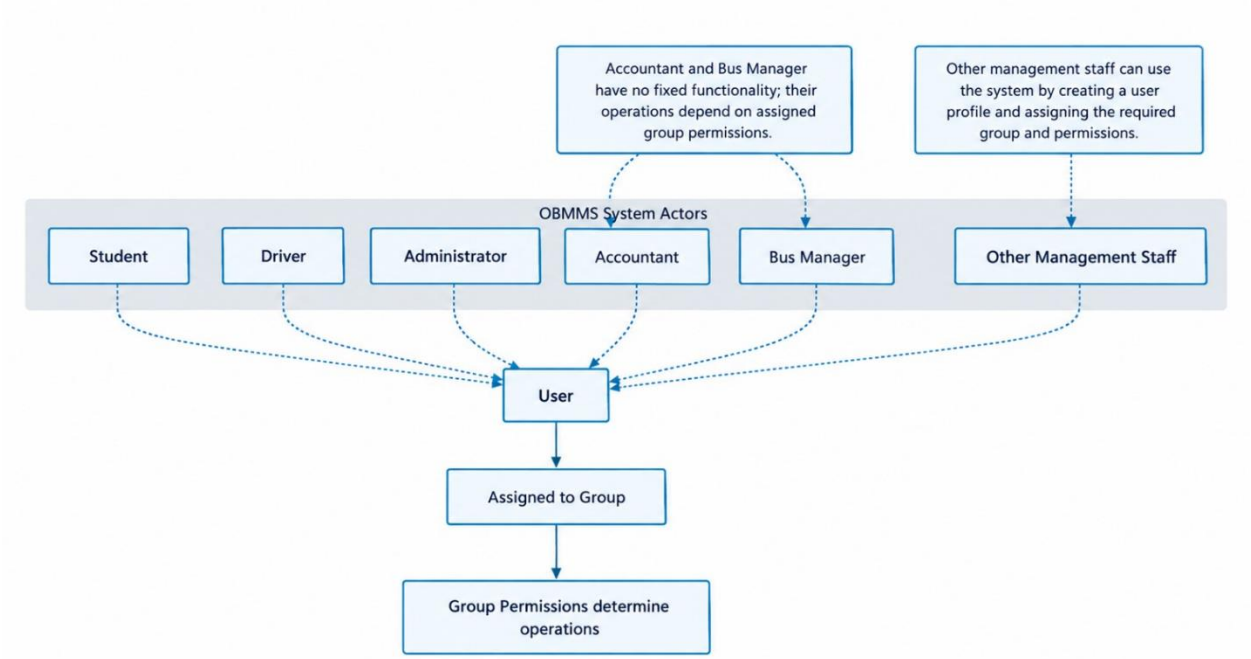


Figure 3.1: System Actors and Their Interaction with OBMMS

The actor structure demonstrates that OBMMS uses a flexible permission-based access model instead of depending only on fixed user roles.

3.3 Use Case Diagram

The use case diagram represents the major interactions between the system actors and OBMMS. It illustrates how different users interact with the system in accordance with their permissions. The use case includes such features as authentication, viewing the transportation information, managing the resources of the system, managing packages, and real-time bus location sharing. Since the system uses group-based permissions, the actual availability of a particular operation depends on the permissions assigned to the user's group.

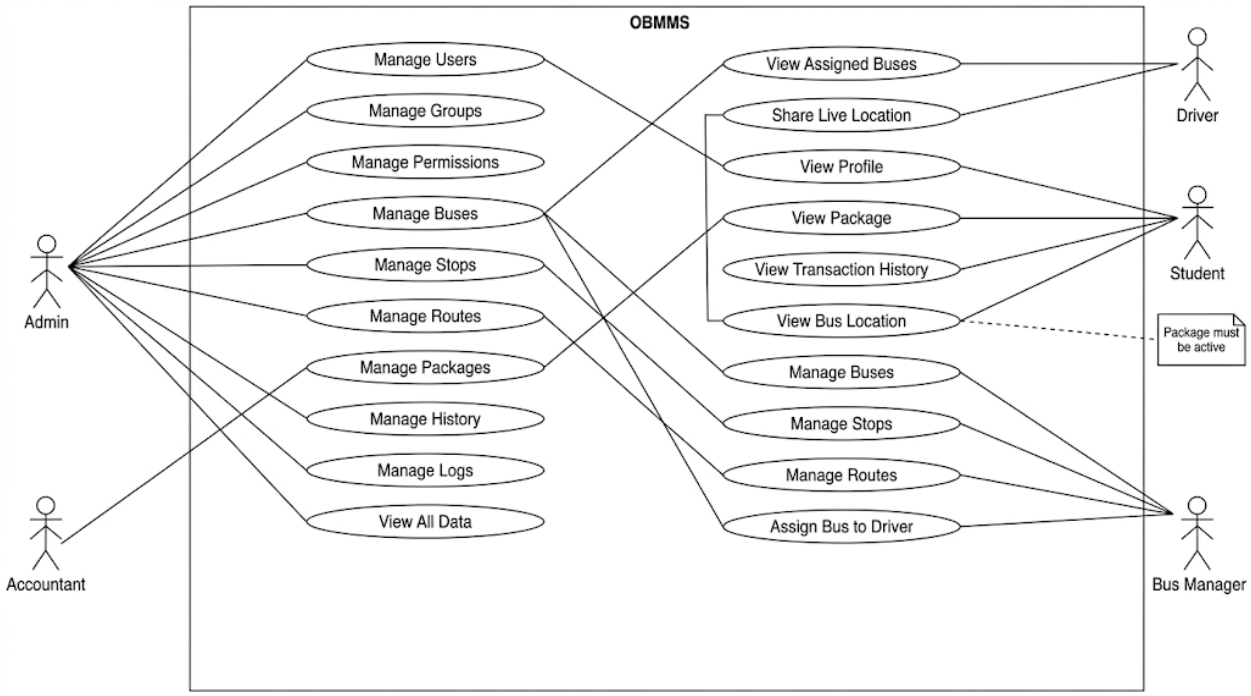


Figure 3.2: Use Case Diagram of OBMMS

3.4 Use Case Descriptions

The major use cases of OBMMS are described below:

3.4.1 User Login

Users provide their username and password to authenticate themselves. After successful authentication, the system generates an authentication token. The user's group and permissions are then used to determine which system resources and operations can be accessed.

3.4.2 User and Group Management

Authorized users can create, view, update, and delete user records according to their permissions. Groups can also be created and configured with different permissions. The group-based permission mechanism allows the system administrator to control access without creating separate hard-coded roles for every possible management responsibility.

3.4.3 Bus Management

Authorized users can manage bus records through CRUD operations. Bus information includes details such as license, capacity, passenger count, assigned driver, location, and active status.

3.4.4 Stop Management

Authorized users can create and manage bus stops. A stop contains information such as its name, description, latitude, longitude, campus status, and active status.

3.4.5 Route Management

Authorized users can manage routes. A route is associated with stops and a bus. The bus is associated with a driver. Therefore, the transportation relationship can be represented as:

Route → Stops → Bus → Driver

A driver may be assigned to more than one bus.

3.4.6 Package Management

Packages provide controlled access to transportation services. Users with the required package permissions can manage package records. The Accountant can activate or deactivate packages if the Accountant's assigned group has Package Read and Write permission.

3.4.7 Real-Time Bus Tracking

A driver logs into the mobile application and selects a bus. The driver's android phone gets the current GPS location and sends it to the backend via WebSocket. The backend processes the location and provides the real-time data to connected and authorized students.

3.4.8 Location Access

Location access is controlled with group permissions and package status. If a user has the needed location Read permission, the user can access the available location information given his or her assigned permissions. If the user does not have location Read permission, the user must have an active package to access bus tracking.

3.4.9 History Management

The system stores history of any package-related transaction. History can be accessed based on the permissions given to the user's group. The deletion of a history is only allowed for an administrator. No other group is permitted to delete history records.

3.5 System Workflow

The overall workflow of OBMMS begins with the user authentication process. A user enters the login credentials to the system. Upon successful authentication, the system identifies the user group and checks the permissions given to the user group.

<u>The overall workflow is as follows:</u>

User → Login → Authentication → JWT Token → Group Identification → Permission Verification → Access to Authorized Features

If a user is found to have the necessary permission, the requested operation is allowed. Otherwise, the operation is denied. For management users, the system provides access based on the permissions assigned to the user group. This enables different university management members to perform their duties using the same system. For students, the mobile application

provides transportation-related information based on their access rights and package status. For drivers, the workflow also involves selecting a bus and sharing real-time GPS location.

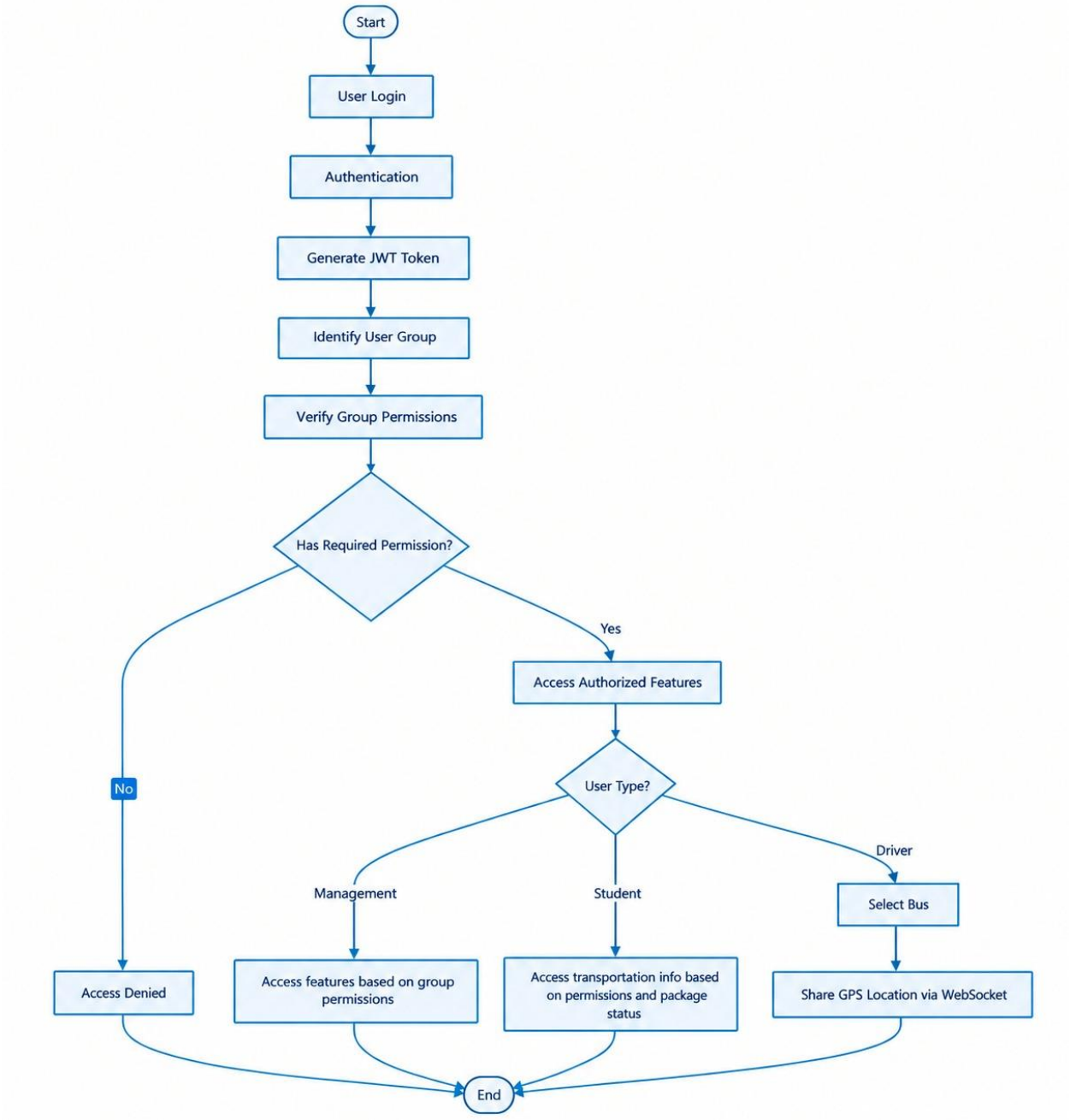


Figure 3.3: General System Workflow

3.6 Real-Time Tracking Workflow

Real-time tracking is one of the main functions of OBMMS. It allows students to see the current location of a bus while the driver is sharing its GPS location. The process begins when the driver logs into the mobile application. The driver then chooses one of the buses assigned to

them. After choosing the bus, the application gets GPS information from the Android device. The location information is sent to the FastAPI backend using a WebSocket connection. The backend manages the active real-time connection and forwards the updated location information to authorized connected users. The student mobile application receives the location information and displays the bus position on the map using OpenStreetMap.

The real-time workflow can be represented as:

Driver → Android Device → GPS Location → WebSocket → FastAPI Backend → Authorized Students → Mobile Application → Map

REST APIs are used for normal request-response operations such as CRUD operations, while Web-Sockets are used for real-time bus location communication. When the driver logs out or the WebSocket connection is disconnected, real-time location sharing stops. The system therefore does not continue receiving live GPS updates from that disconnected driver. When the driver logs out or the WebSocket connection is disconnected, real-time location sharing stops. The system therefore does not continue receiving live GPS updates from that disconnected driver.

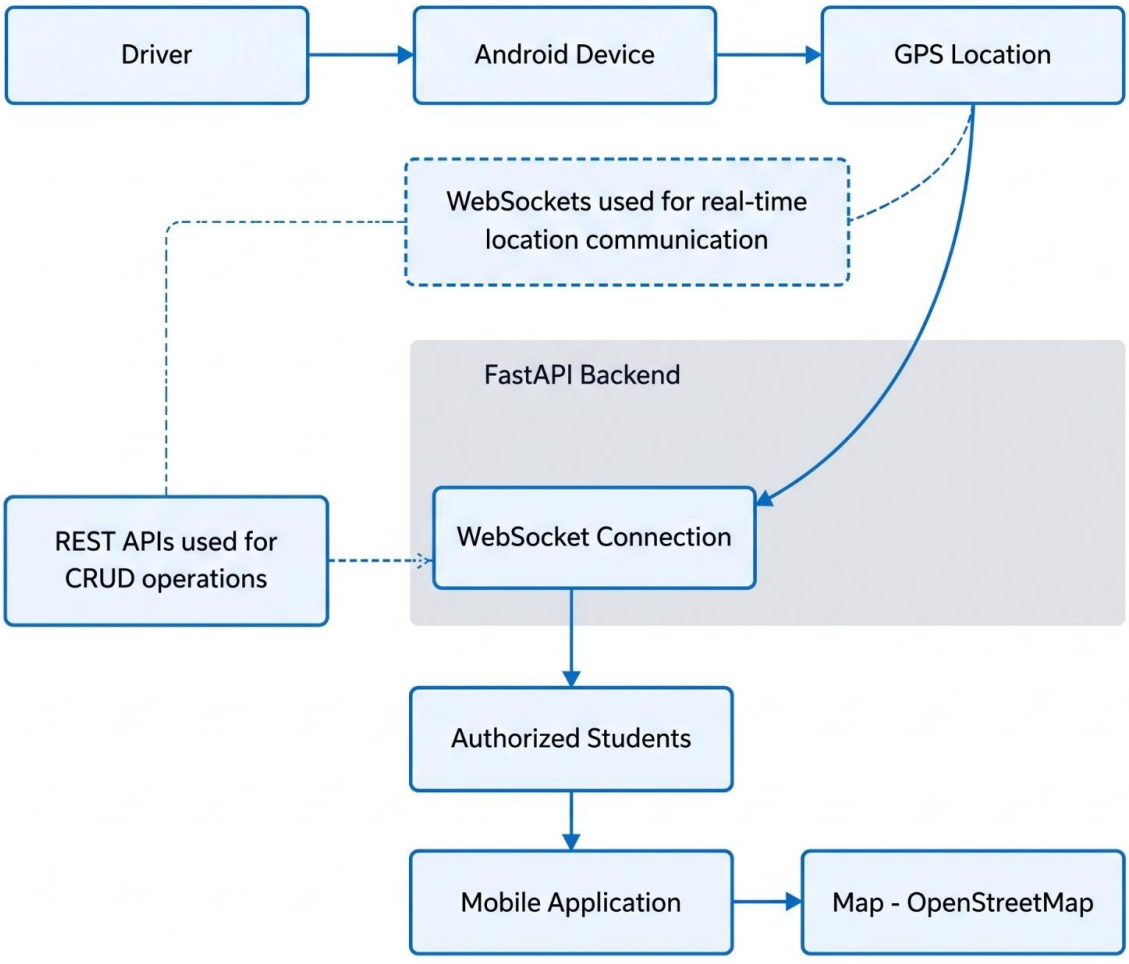


Figure 3.4: Real-Time Bus Tracking Workflow

3.7 Package Access Workflow

Package access is used to control access to transportation services and bus tracking. A student first logs into the system and requests access to transportation information. The system checks the user's permissions and package status. If the user has the required location Read permission, location information can be accessed according to the assigned permissions. If the user does not have location Read permission, an active package is required for accessing bus tracking.

The basic workflow is:

- **Student → Login → Permission Check → Location Read Permission?**
- **Yes → Access Location According to Permission**
- **No → Check Active Package → Package Active → Allow Tracking**
- **No Active Package → Tracking Access Not Available**

Package activation and deactivation are performed by an authorized user whose group has Package Read and Write permission. In the current system, this operation can be performed by the Accountant when the required permission has been assigned to the Accountant's group. The package system therefore provides an additional access-control mechanism for transportation services.

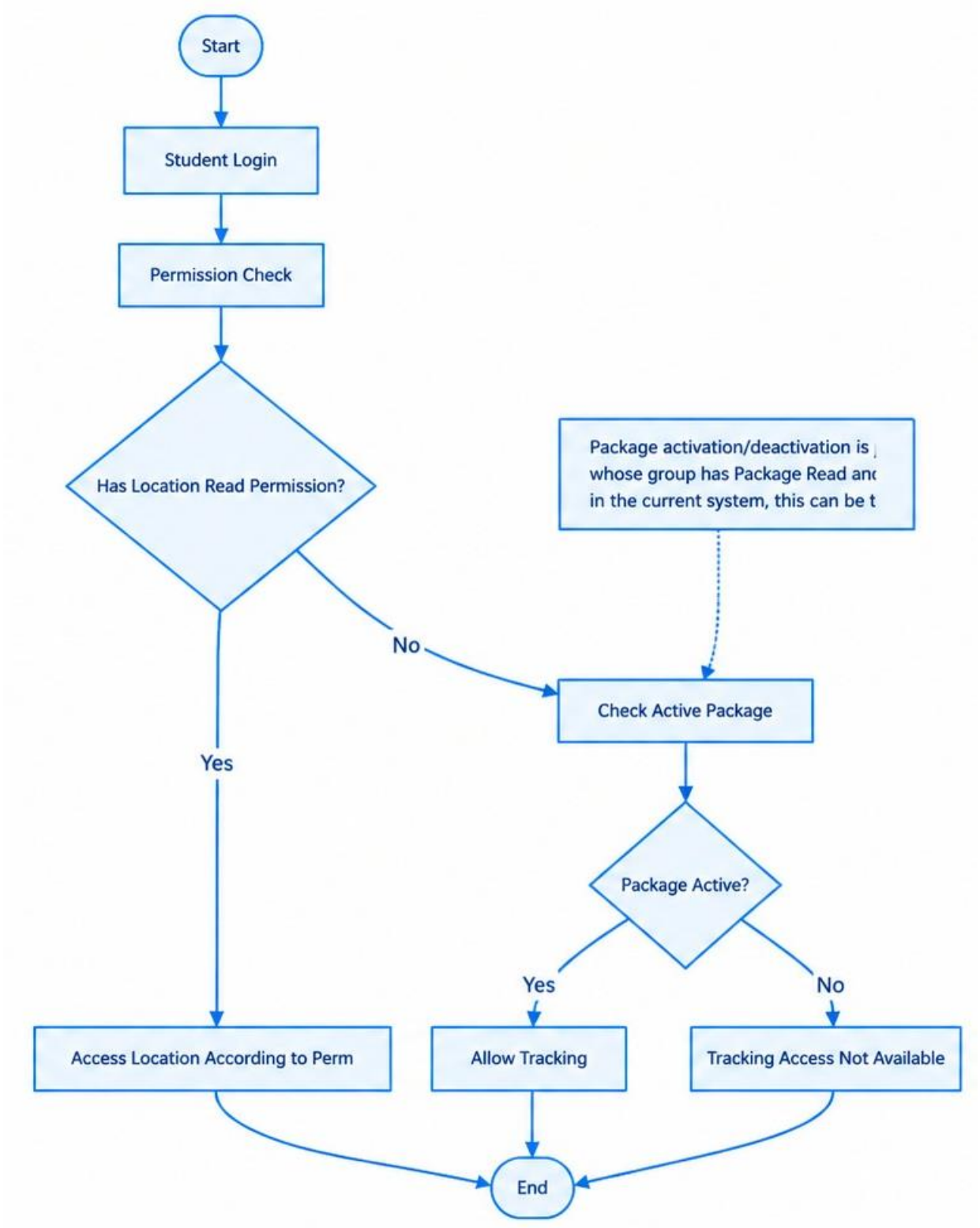


Figure 3.5: Package Access Workflow

Chapter 4
System Design

Chapter 4: System Design

This chapter presents the design of the Online Bus Monitoring and Management System (OBMMS). The system design describes the architecture, database structure, domain model, application programming interface, and the various behavioral models that collectively define how the system is constructed. The design conforms to the client-server architecture where FastAPI monolithic backend serves ReactJS web administration panel and React Native mobile application for students and drivers. The design is presented with the use of standard notations and diagrams used in the software engineering discipline.

4.1 System Architecture

The system architecture of OBMMS adopts a three-tier client–server model. The three tiers are the presentation tier, the application tier, and the data tier. This separation of concerns enables the system to be modular, manageable, and scalable. The presentation tier consists of two distinct client applications. The first one is a web-based administration panel built using ReactJS, which is used by the administrators and management staff to perform CRUD operations on system resources. The second one is a React Native mobile application used by students and drivers. Students use the mobile application to view buses, routes, stops, packages, and real-time bus locations. Drivers use the mobile application to select an assigned bus and share its GPS location with the backend.

The application tier consists of a FastAPI monolithic backend. The backend is in charge of all business logic, authentication, authorization, permission check, and data processing. The backend exposes REST API endpoints for standard request–response operations such as CRUD operations and authentication. In addition, the same FastAPI backend is hosting a WebSocket server that is used for real-time bidirectional communication between drivers and students. The WebSocket server handles the transmission of live GPS location data from the driver devices to the backend and the subsequent broadcasting of that location data to the authorized connected students. The data tier consists of a MySQL relational database. The database is storing all persistent system data such as users, groups, permissions, buses, routes, stops, locations, packages, history records, logs, and notifications. SQLAlchemy is used as the Object-Relational Mapping (ORM) layer to interact with the database from the FastAPI backend. Authentication is achieved using JSON Web Tokens (JWT). Upon a user login, the backend will generate a JWT token that the client includes in subsequent requests. The token will have information about the user identity, and it will be used by the backend to check permissions before granting access to any protected resources.

The overall architecture can be summarized as:

- **Presentation Tier:** ReactJS web application (admin/management) and React Native mobile application (students/drivers)

- **Application Tier:** FastAPI monolithic backend with REST API and WebSocket server
- **Data Tier:** MySQL relational database accessed through SQLAlchemy ORM

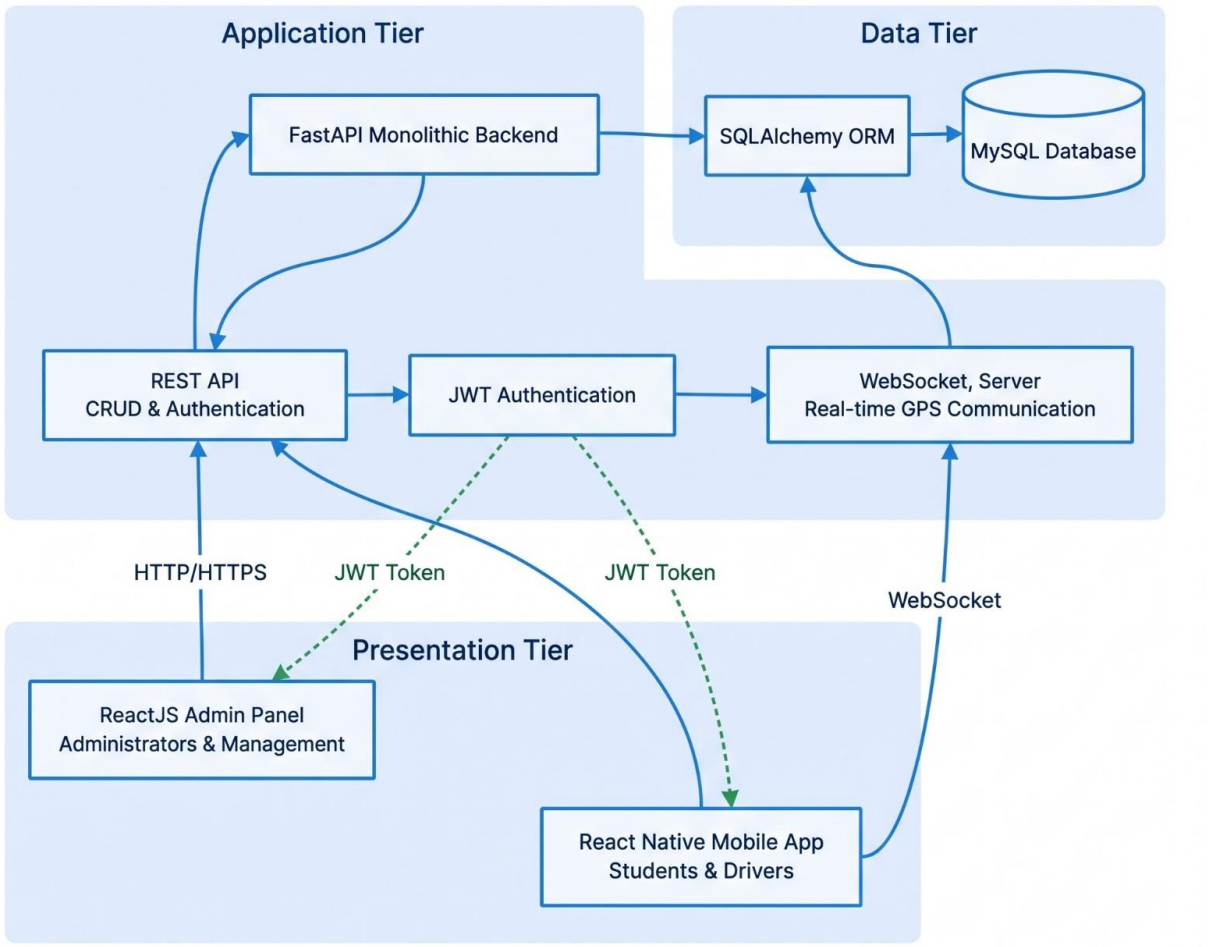


Figure 4.1: System Architecture of OBMMS

4.2 Application Architecture

The application architecture defines how the diverse software components of OBMMS are organized and how they interact with one another. The backend is designed as a monolithic FastAPI application. All modules — including authentication, user management, group management, bus management, route management, stop management, location management, package management, history management, logging, and notification — are part of a single deployable unit. It is built using a layered architectural pattern. The layers are the routing layer, the service layer, the data access layer, and the database layer. The routing layer defines the REST API endpoints and WebSocket endpoints. The service layer contains the business logic and permission verification. The data access layer uses SQLAlchemy models to interact with the MySQL database. The database layer consists of the MySQL server and the physical tables.

The web administration panel communicates solely with the backend through REST-API endpoints. The mobile application communicates with the backend through REST API endpoints for authentication, CRUD operations, and data retrieval, and through the WebSocket endpoint for real-time bus location tracking. The permission-based access control is a central feature of the application architecture. Every user belongs to exactly one group. Each group has a set of permissions stored as a JSON field. The permissions define the level of access — Not Allowed, Read Only, or Read and Write — for each system resource. The backend verifies these permissions for every request before allowing the operation to proceed.

The application architecture can be described as follows:

- **Client Layer:** ReactJS web application and React Native mobile application
- **API Layer:** FastAPI REST endpoints and WebSocket endpoint
- **Business Logic Layer:** Service modules for each domain entity
- **Data Access Layer:** SQLAlchemy ORM models
- **Database Layer:** MySQL relational database

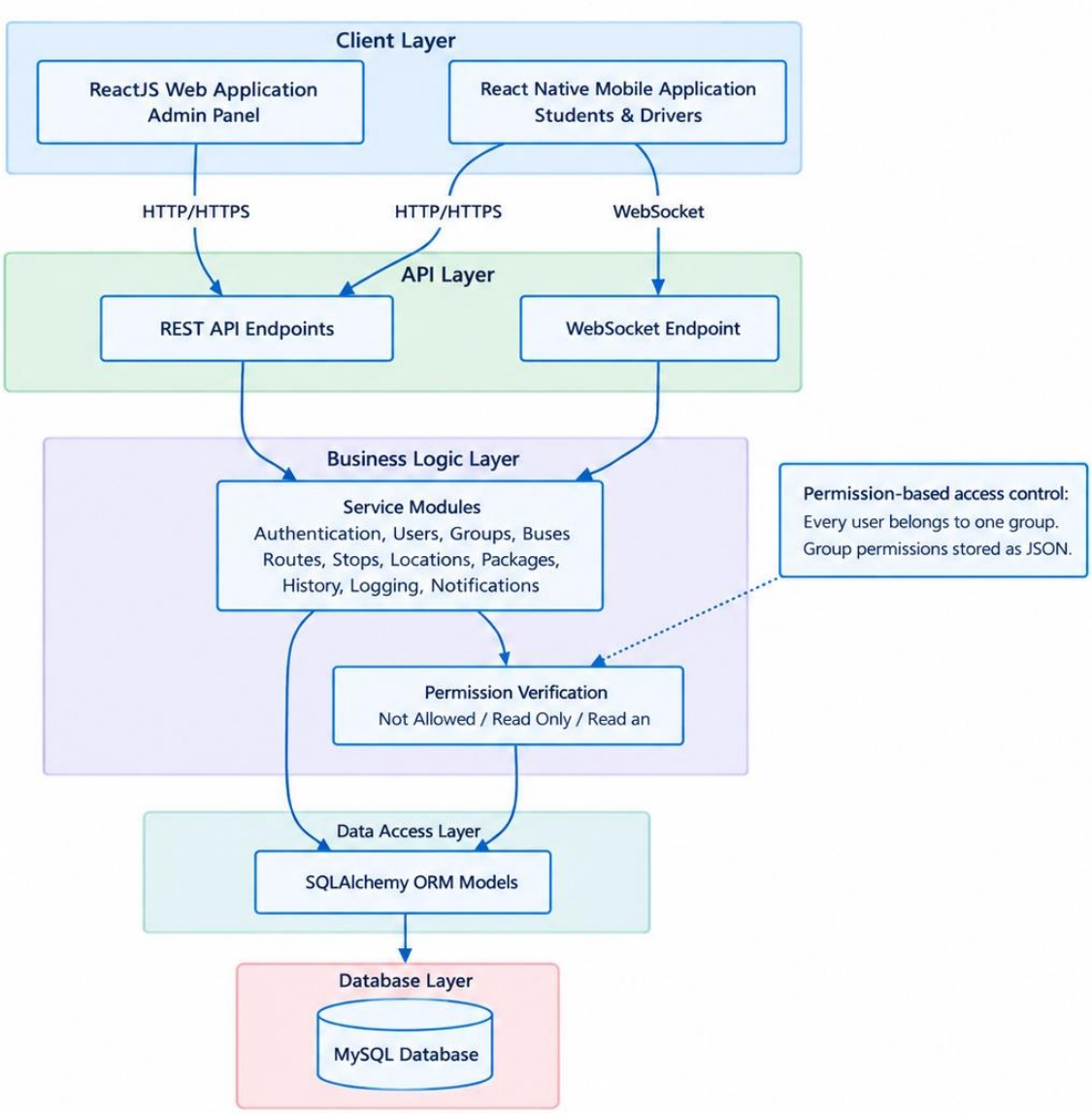


Figure 4.2: Application Architecture of OBMMS

4.3 Database Design

The database design of OBMMS is presented by an Entity – Relationship (ER) diagram and a data dictionary. The database is implemented by MySQL and accessed by SQLAlchemy ORM. The database consists of eleven tables: users, group, bus, location, stop, route, package, history, logs, and notifications. The relationships between the tables are identified by the foreign key references. The users table is associated with the group table by the group field. The bus table is related to the users table by the driver field. The route table is linked to the bus table by the bus field. The route table is also related to the stop table by the departure and destination fields. The package table is connected to the users table by the user field. The history table is related to the package table by the package field and associated with the users table by the user field.

4.3.1 ER Diagram

The ER diagram below represents the entities of the OBMMS database and the relationships between them. The main entities are Users, Group, Bus, Location, Stop, Route, Package, History, Logs, and Notifications. The Users entity has a one-to-one relationship with the Group entity. Each user belongs to exactly one group. The Group entity contains a permissions field that stores the access rights for all users in that group. The Bus entity has a many-to-one relationship with the Users entity through the driver field. A driver can be assigned to multiple buses, but each bus has one assigned driver. The Route entity has a many-to-one relationship with the Bus entity. Each route is associated with one bus, and a bus can be associated with multiple routes. The Route entity also has a relationship with the Stop entity through the departure and destination fields. Each route has a departure stop and a destination stop.

The Package entity has a many-to-one relationship with the Users entity. Each package belongs to one user, and a user can have multiple packages. The History entity has a many-to-one relationship with both the Package entity and the Users entity. Each history record is associated with one package and one user. The Location entity stores GPS location data including latitude, longitude, heading, and speed. This entity is used for real-time tracking and historical location data. The Logs entity stores system activity logs including the user, the model affected, the action performed, the row affected, date, and time.

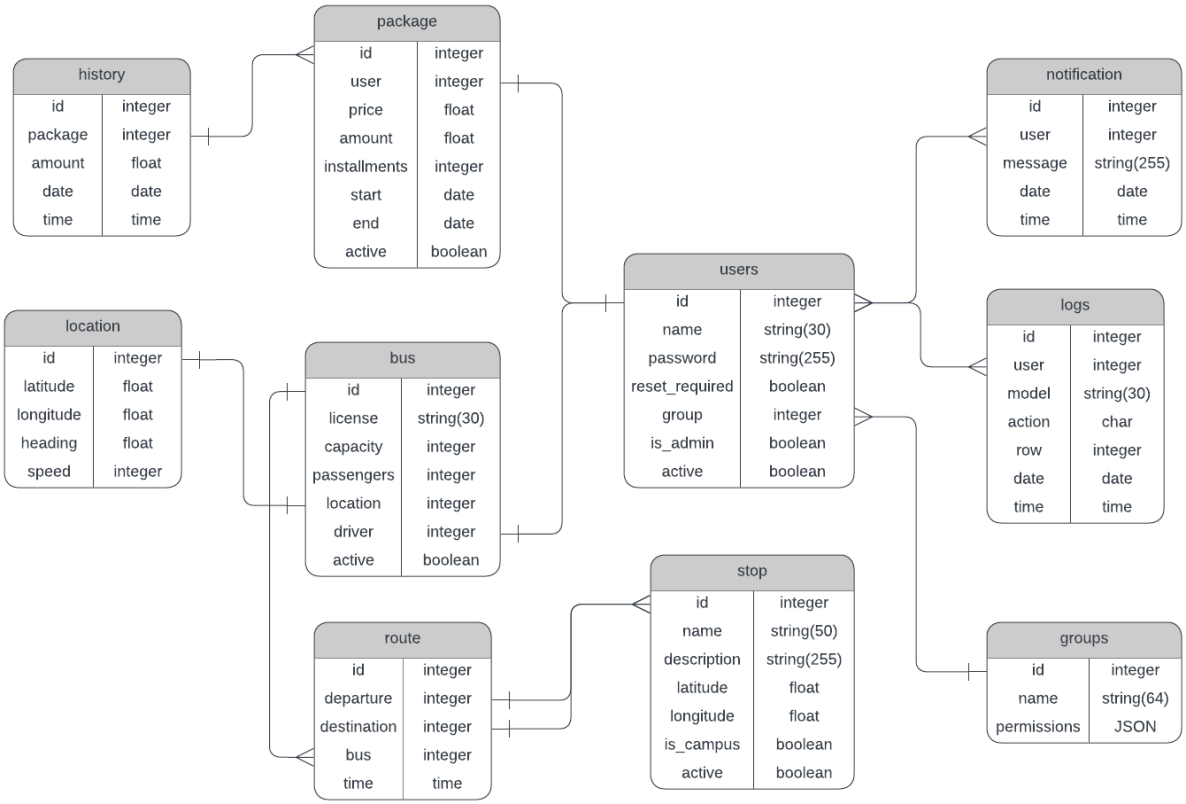


Figure 4.3: ER Diagram of OBMMS Database

4.5 Class Diagram

The class diagram of OBMMS represents the software classes that implement the domain model. The class diagram includes the attributes and methods of each class, as well as the relationships between classes. The main classes are Users, Group, Bus, Location, Stop, Route, Package, History, Logs, and Notifications. These classes corresponded to the SQLAlchemy models defined in the backend. The Users class has the attributes of id, name, password, reset_required, group, is_admin, and active. It provides a method for password verification and serialization. The Group class has the attributes of id, name, and permissions. The permissions attribute is a JSON object that defines the access level for each resource.

- The Bus class has the attributes of id, license, capacity, passengers, location, driver, and active.
- The Location class has attributes id, latitude, longitude, heading, and speed.
- The Stop class has attributes id, name, description, latitude, longitude, is_campus, and active.
- The Route class has attributes id, departure, destination, bus, time, and active.
- The Package class has attributes id, user, price, month, year, and active.
- The History class has attributes id, package, user, amount, date, and time.
- The Logs class has attributes id, user, model, action, row, date, and time.
- The Notifications class has attributes id, user, message, date, and time.

The class diagram also includes the relationships between classes. The Users class has a one-to-one relationship with the Group class. The Bus class has a many-to-one relationship with the Users class through the driver attribute. The Route class has a many-to-one relationship with the Bus class. The Package class has a many-to-one relationship with the Users class. The History class has a many-to-one relationship with both the Package and Users classes.

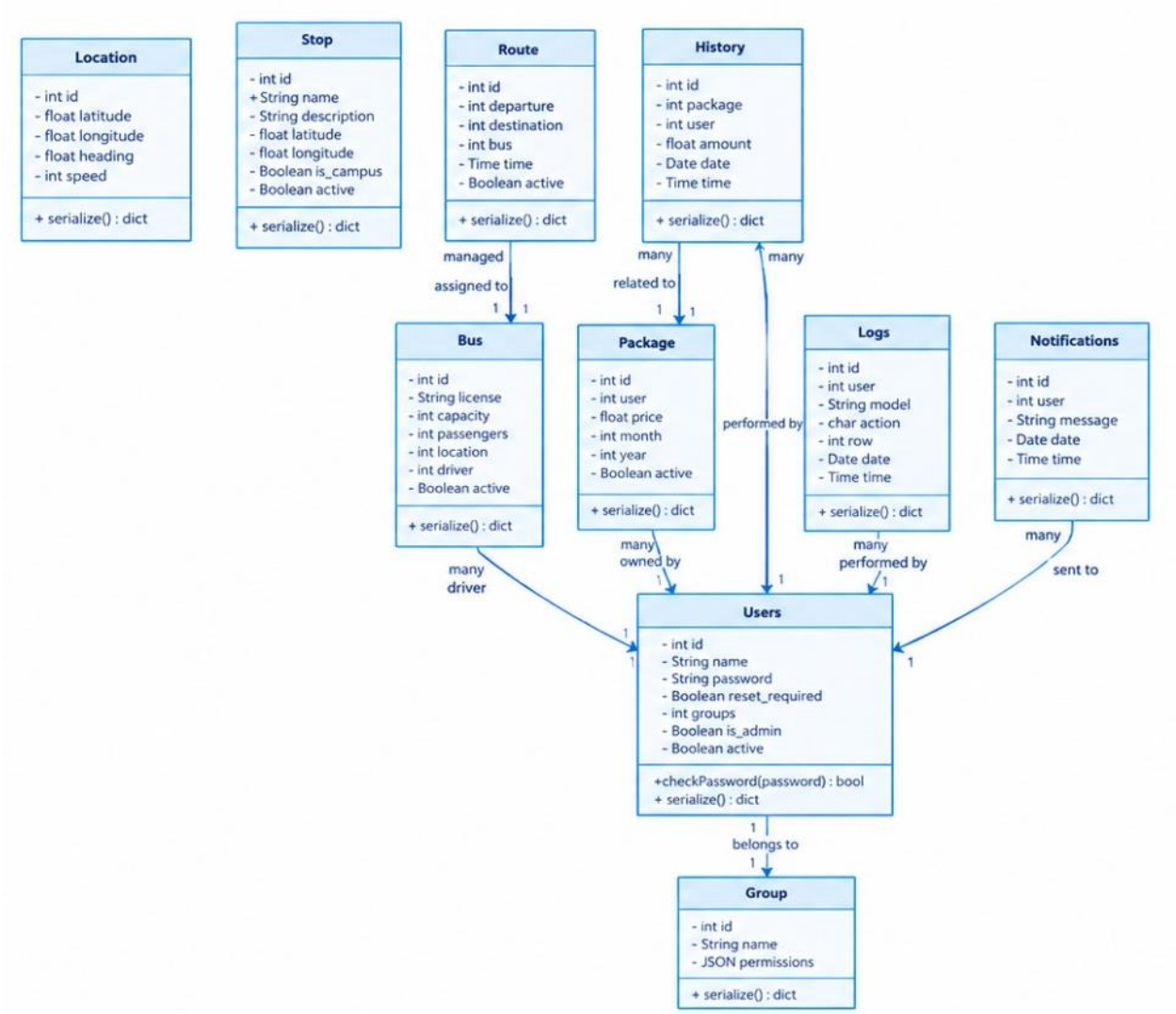


Figure 4.5: Class Diagram of OBMMS

4.6 Sequence Diagrams

User Login Sequence

The user starts the login process and sends the requested user credentials to the backend via mobile or web application. The backend checks the provided credentials against the Users table and approves the request if the provided combination matches a user in the database. The backend then creates a JWT Token and sends it back to the client application, which stores it for further use in the session. The backend also queries the users group and permissions and embeds them in the token or the response.

<u>The sequence of activities is:</u>

1. The User enters the provided username and password
2. The Application sends a POST /login request with the provided credentials to the Backend

3. The Backend asks the database for a user with a matching username
4. The Database returns the user
5. The Backend checks if the provided password matches the hash of the retrieved user
6. The Backend generates JWT Token
7. The Backend returns the token to the application
8. The Application displays the authorized interface to the user

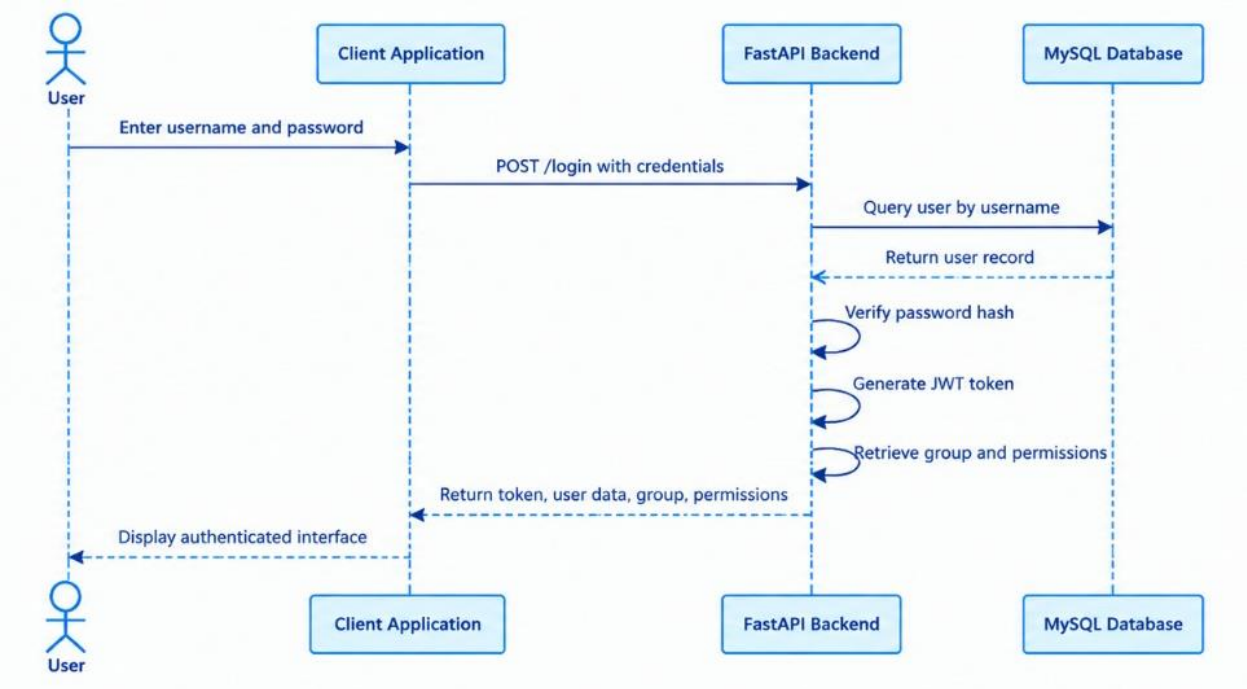


Figure 4.6: Sequence Diagram for User Login

Real-Time Bus Tracking Sequence

The driver logs into the mobile application and chooses an assigned bus to start tracking in real-time. The mobile application acquires the current device location using GPS and sends it to the backend via a WebSocket. The backend stores the location in the database, and all the connected and authorized students receive the data via WebSockets. The student application receives the data and updates the bus position on the map.

<u>The sequence of activities is:</u>

1. The Driver logs into the Mobile Application
2. The Mobile Application requests a WebSocket connection to the Backend
3. The Backend establishes a connection to the Mobile Application
4. The Driver selects a bus from the list
5. The Mobile Application requests the current device location from the Device GPS
6. The Device GPS returns the current latitude, longitude, heading, and speed
7. The Mobile Application sends the location data to the Backend via WebSockets

8. The Backend stores the received location data in the database
9. The Backend sends the location data to the Student Mobile Application via WebSockets
10. The Student Mobile Application updates the bus position on the map

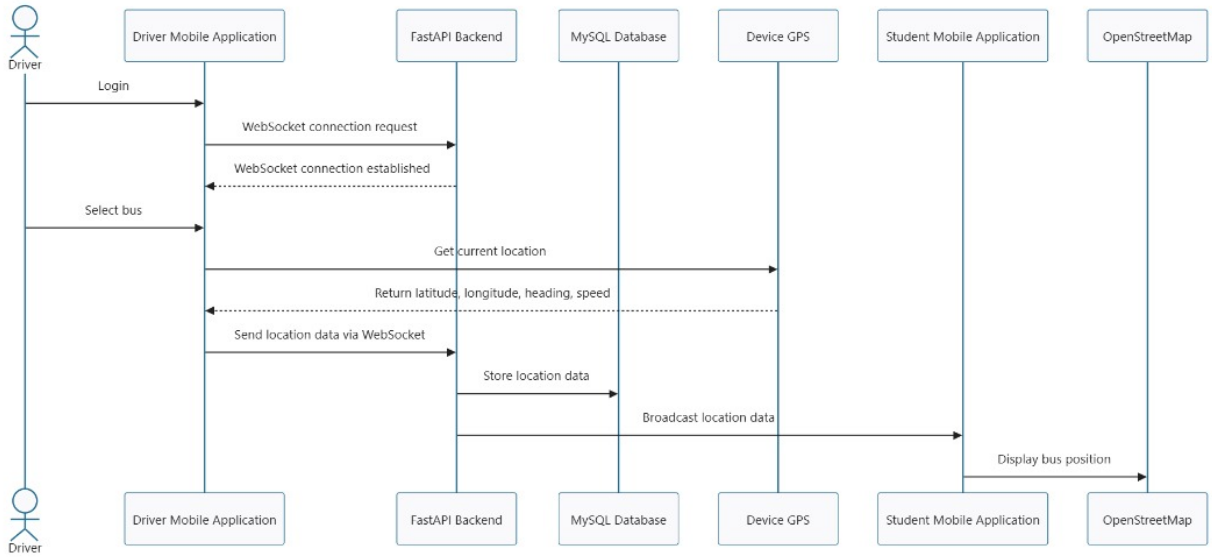


Figure 4.7: Sequence Diagram for Real-Time Bus Tracking

Package Activation Sequence

The accountant logs into the application and accesses the package management menu to activate or deactivate the package. The backend checks if the account belongs to a group with Package Read and Write permissions and updates the flag if it does. The system may also store a history of the changes in a history table.

<u>The sequence of activities is:</u>

1. The Accountant logs into the Client Application
2. The Client Application authenticates the user and returns the token
3. The Accountant navigates to the package management section
4. The Client Application sends a request to the Backend to receive a list of packages
5. The Backend queries the list of packages from the Database
6. The Backend returns the list of packages to the Client Application
7. The Accountant selects the package to activate or deactivate
8. The Client Application sends an update request to the Backend
9. The Backend checks if the Accountant has Package Read and Write permission
10. The Backend updates the active flag of the package in the Database
11. The Backend writes the history of changes into the Database
12. The Backend returns a success response to the Client Application

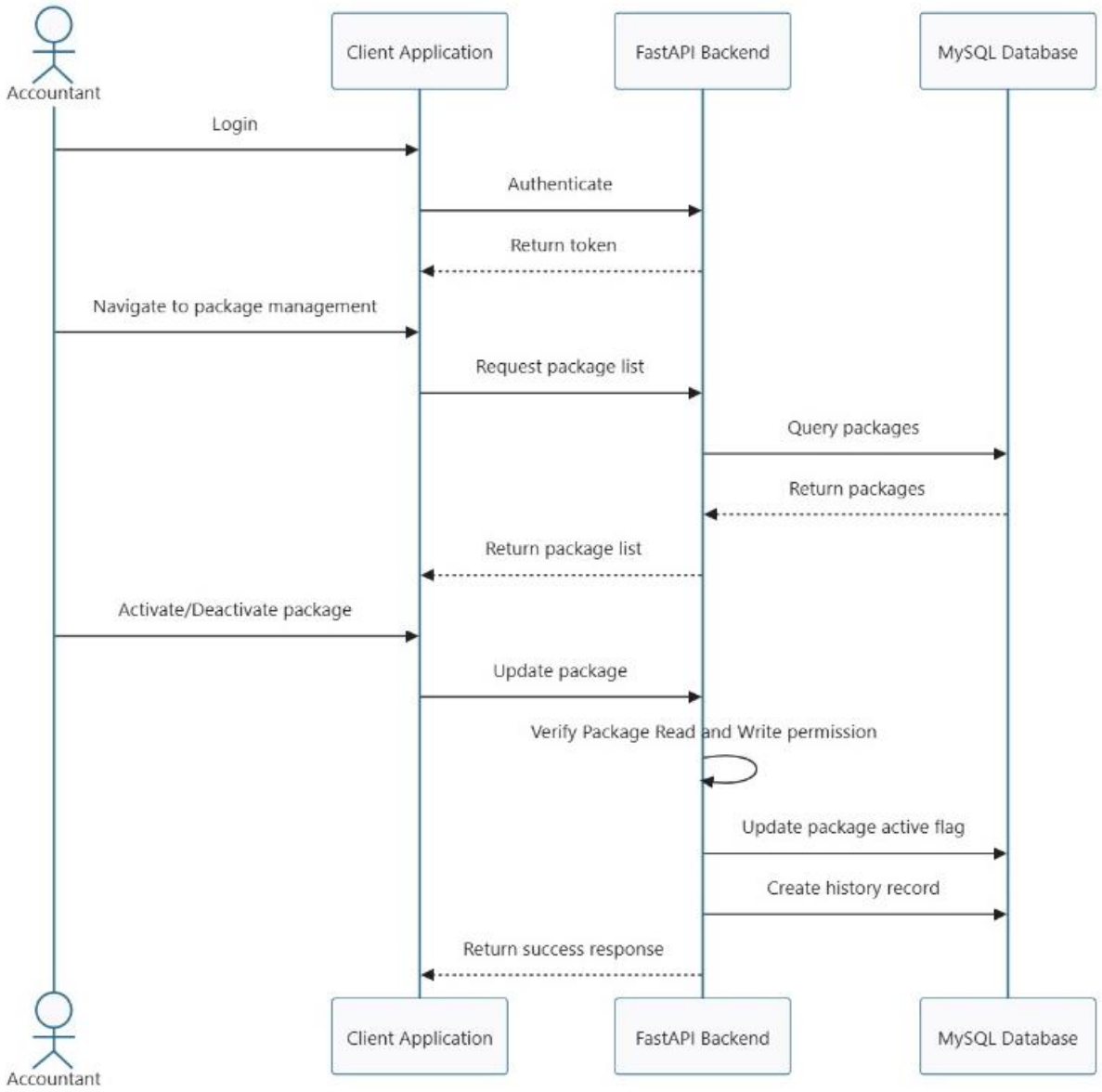


Figure 4.8: Sequence Diagram for Package Activation

4.7 Activity Diagrams

Activity diagrams depict the flow of control for the major processes of OBMMS. The main activity diagrams are described below.

Login Activity

The login activity begins when the user opens the application and enters credentials. The system validates the credentials against the database. If the credentials are valid, the system generates a JWT token and grants access. If the credentials are invalid, the system displays an error message and prompts the user to try again.

The activity flow is:

Start → Enter credentials → Submit → Validate credentials → [Valid?] → Yes: Generate JWT → Grant access → End

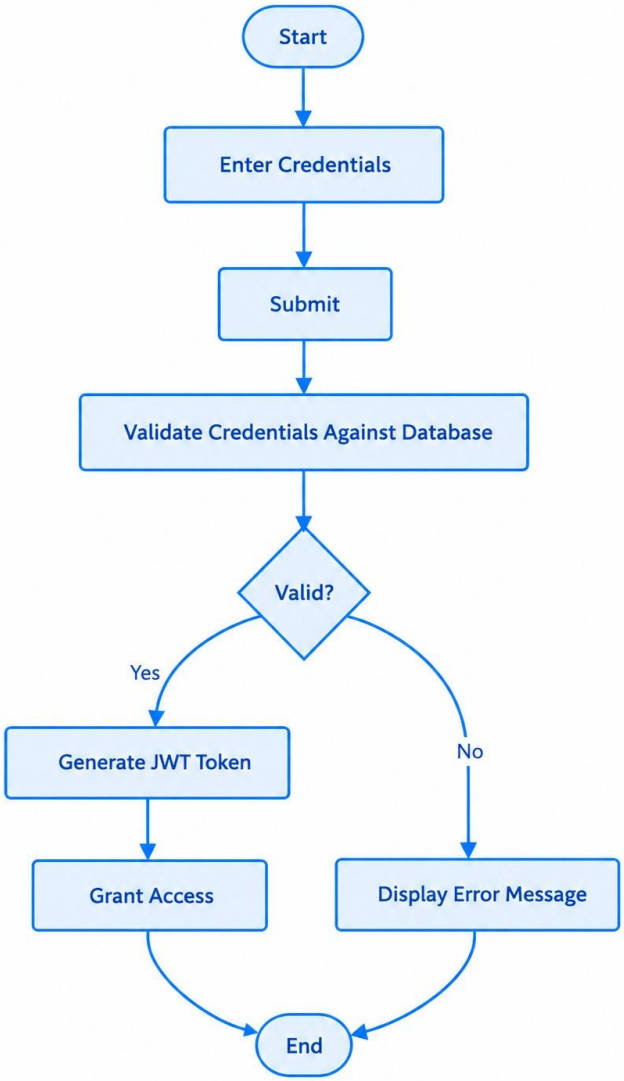


Figure 4.9: Activity Diagram for Login

Bus Tracking Activity

The application uses the activity below when the user logs in as a driver, selects a bus, and shares his or her location. The application gets the GPS location and connects to the WebSocket server. It sends GPS data until the session ends or the driver stops sharing the location. Students get this data and see the bus location on the map.

The activity flow is as follows:

Start → Driver login → Driver selects the bus → Get GPS location → Connect to WebSocket server → Send GPS data → [Continue?]

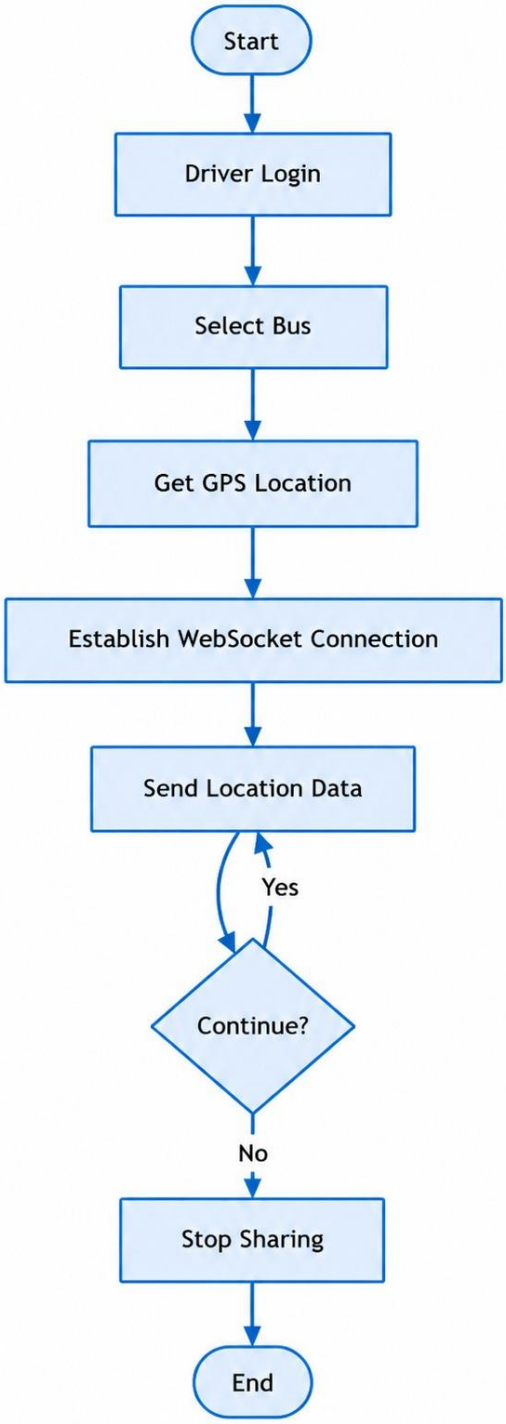


Figure 4.10: Activity Diagram for Bus Tracking

Package Access Activity

The package access activity occurs when a student requests a bus tracking access. The system evaluates whether the student's group has a Location Read permission or not. In case of positive response, the access is granted. Otherwise, it checks whether the student has an active package or not. In case of positive response, the access is granted. Otherwise, it is denied.

The activity flow is:

Start → Student requests tracking → Check Location Read permission → [Permission?] → Yes: Grant access → End

[Permission?] → No: Check active package → [Active package?] → Yes: Grant access → End

[Active package?] → No: Deny access → End

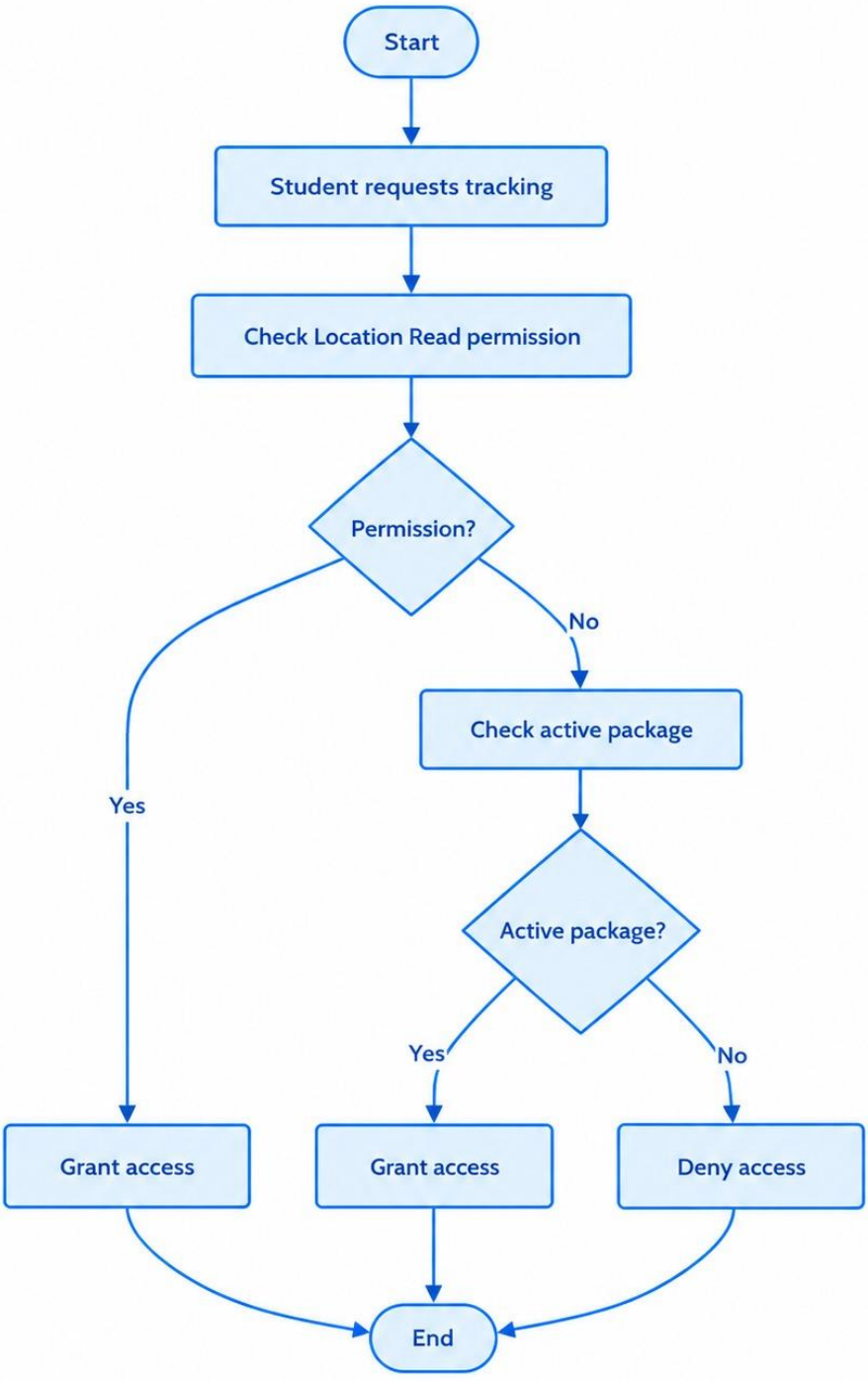


Figure 4.11: Activity Diagram for Package Access

4.8 State Transition Diagram

State transition diagrams model the states of specific system entities and the events that cause transitions between states. The main state transition diagrams for OBMMS are described below.

Bus State Transition

A bus can be in one of two states: Active or Inactive. When a bus is in service, it is in the Active state. When a bus is not in service, it is in the Inactive state. The transition from Active to Inactive occurs when the bus is taken out of service. The transition from Inactive to Active occurs when the bus is put into service.

States: Active, Inactive
Transitions: Active → Inactive (when bus is taken out of service), Inactive → Active (when bus is put into service)

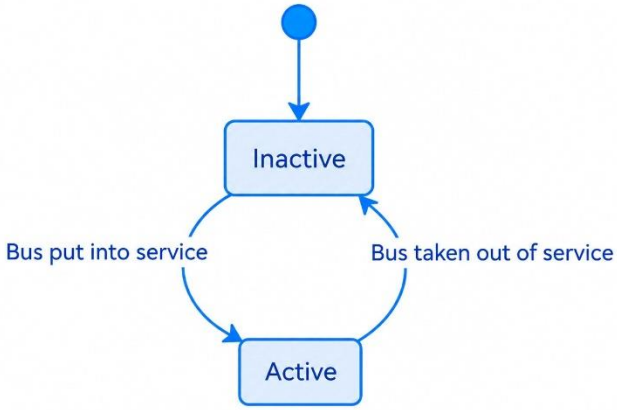


Figure 4.12: State Transition Diagram for Bus

Driver Tracking State Transition

A driver can be in one of two tracking states: Sharing Location or Not Sharing Location. When the driver logs in and selects a bus, the state transitions from Not Sharing Location to Sharing Location. When the driver logs out or stops sharing, the state transitions back to Not Sharing Location.

- **States:** Not Sharing Location, Sharing Location
- **Transitions:** Not Sharing → Sharing (driver selects bus and starts sharing), Sharing → Not Sharing (driver stops sharing or logs out)

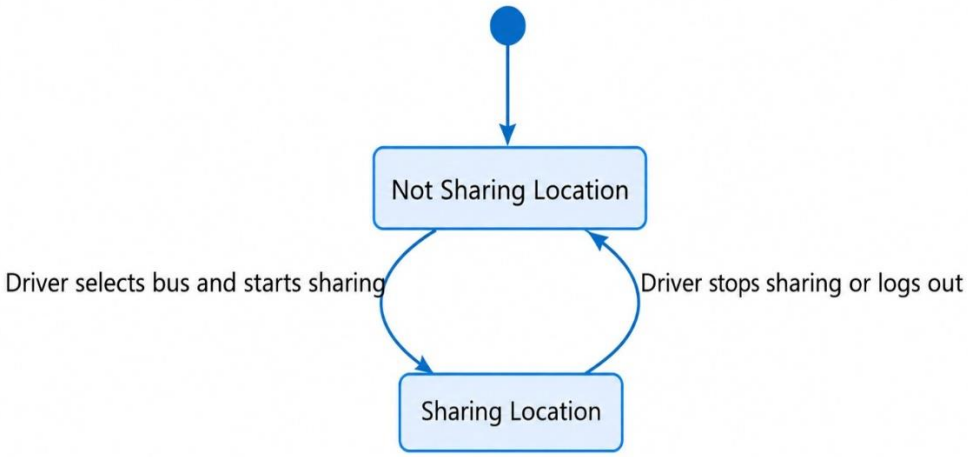


Figure 4.13: State Transition Diagram for Driver Tracking

Package State Transition

A package can be in one of two states: Active or Inactive. When a package is activated by an authorized user, the state transitions from Inactive to Active. When the package expires or is deactivated, the state transitions from Active to Inactive.

- **States:** Inactive, Active
- **Transitions:** Inactive → Active (package activated), Active → Inactive (package deactivated or expired)

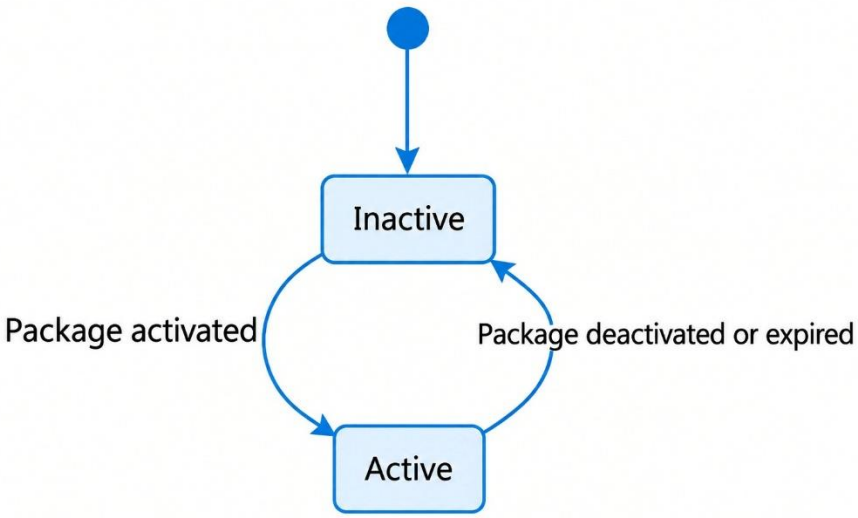


Figure 4.14: State Transition Diagram for Package

4.9Component Diagram

The component diagram below represents the software components of OBMMS and their dependencies. The system is divided into 5 components including The Main Project Entry Point, the Admin, the Auth, the Accounts, and the Tracking modules. Each component encapsulates a group of related features and responsibilities, collaborating with each other to achieve the system's objectives.

Main Project Entry Point Module

This is the main FastAPI application that combines all other modules. It is responsible for initializing the application, setting up middleware, and hosting the WebSocket server for real-time communication. It serves as the aggregation point, routing requests to the relevant modules and managing the application's life cycle. It also handles the root and catch-all routes for the web frontend.

Admin Module

The Admin module handles all the CRUD operations for all the system's resources, including users, groups, buses, stops, routes, packages, logs, and history. It provides the endpoints at */api/admin* and requires permission checks from the Auth module before fulfilling any request. The module serves the administrators and management staff.

Auth Module

The Auth module handles the authentication and authorization of the application. It provides the login, logout, change-password, and permissions endpoints at */api/auth*. The module is responsible for generating and verifying JWT tokens and permissions, determining the user's group or permissions. All other modules depend on this one to ensure that a request is permitted before fulfilling it.

Accounts Module

The Accounts module handles packages and history. It provides the endpoints at */api/accounts* for viewing packages, history, and models. It serves accountants and other authorized users to manage packages and view history. It depends on the Auth module to verify permissions and the database to store data.

Tracking Module

The Tracking module handles bus, stops, and routes information, and location tracking. Provides endpoints at */api/tracking* for buses, active-buses, my-buses, stops, location, and routs. It connects to the WebSocket server (hosted by the Main Project Entry Point) to receive and broadcast live GPS data from drivers to students. Depends on the Auth module for permissions and the database to store data.

Dependencies

- The Main Project Entry Point depends on all other modules to provide a unified application.
- The Admin, Accounts, and Tracking modules depend on the Auth module for authentication and permission verification.
- All modules depend on the MySQL database for persistent storage.
- The Tracking module specifically depends on the WebSocket server for real-time communication.
- The Main Project Entry Point hosts the WebSocket server and integrates the other modules.

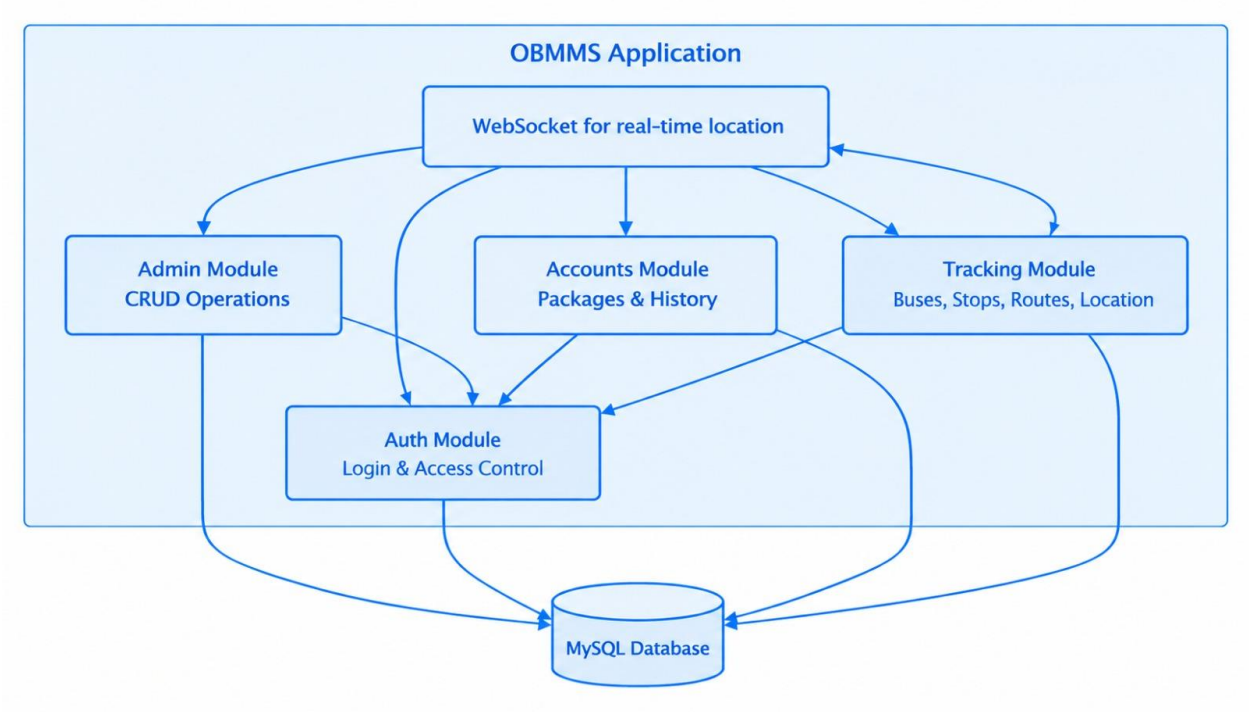


Figure 4.15: Component Diagram of OBMMS

4.10 Deployment Diagram

The deployment diagram depicts the physical deployment of the OBMMS components across hardware nodes. The system is deployed across three main nodes: the client devices, the application server, and the database server. The client devices include smartphones running the React Native mobile application and computers running the ReactJS web application. These devices communicate with the application server over the network. The application server hosts the FastAPI backend, the REST API endpoints, and the WebSocket server. The application server communicates with the database server to store and retrieve data. The database server hosts the MySQL database. The database server is accessible only from the application server.

The deployment configuration can be summarized as:

- **Client Device (Smartphone):** React Native Mobile Application

- **Client Device (Computer):** ReactJS Web Application
- **Application Server:** FastAPI Backend, REST API, WebSocket Server
- **Database Server:** MySQL Database

The communication protocols are HTTP/HTTPS for REST API calls and WebSocket for real-time location communication. The database connection uses the MySQL protocol.

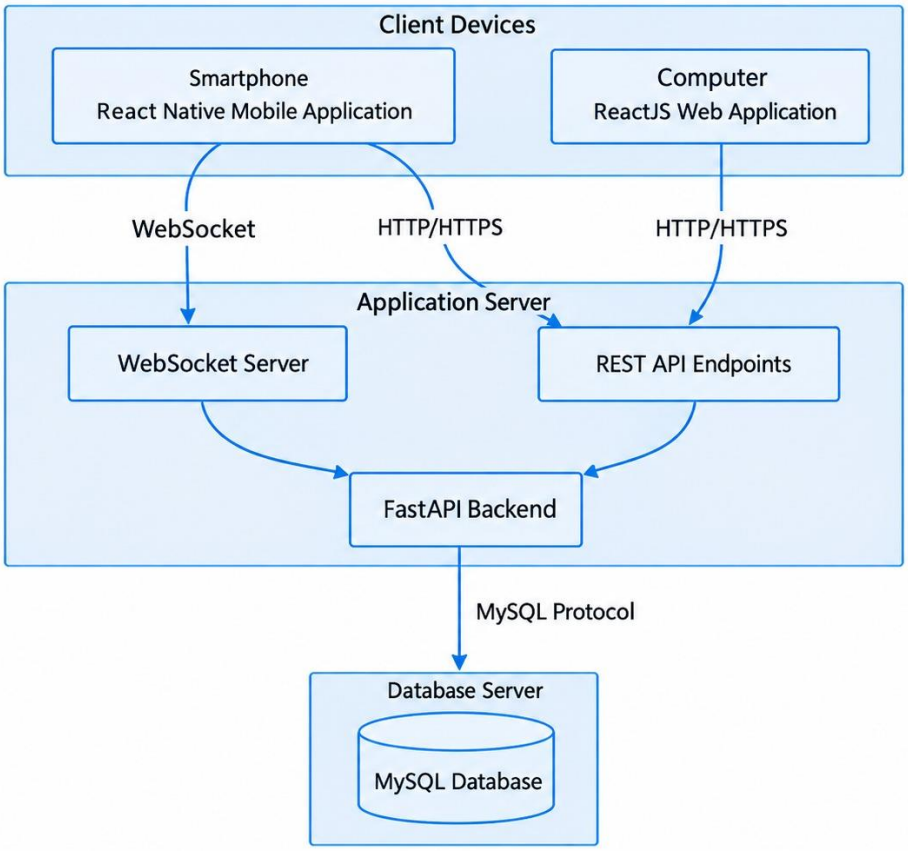


Figure 4.16: Deployment Diagram of OBMMS

4.11 Data Flow Diagram

Level 0 — Context Diagram

The context diagram represents the entire system as a single process and shows the external entities that interact with it. The external entities are Students, Drivers, Administrators, Accountants, and Bus Managers. The data flows between the external entities and the system are as follows:

- Students provide login credentials, request tracking, and inquire about the packages. They also receive bus location, route information, and package status.

- Drivers send login credentials, bus selection, and GPS location data. They receive bus assignment information.
- Administrators send management commands for users, groups, buses, routes, stops, and packages. They receive system data and confirmation messages.
- Accountants send package management commands. They receive package data and confirmation messages.
- Bus Managers send management commands according to their permissions. They receive system data and confirmation messages.

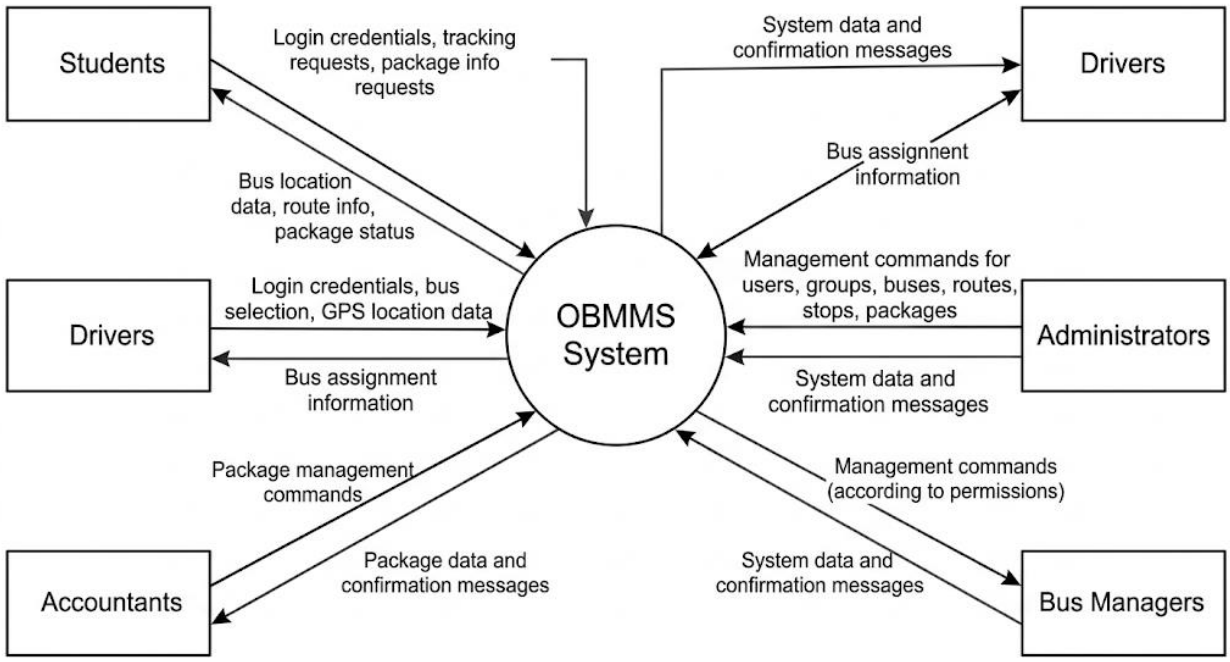


Figure 4.17: Level 0 Data Flow Diagram (Context Diagram)

Level 1 — Top-Level DFD

Level 1 DFD breaks the system into major processes. The main process includes Authentication, User and Group Management, Bus Management, Route and Stop Management, Package Management, Real-Time Tracking, and History Management.

The data stores are:

- Users and Groups (users, group tables)
- Buses (bus table)
- Routes and Stops (route, stop tables)
- Locations (location table)
- Packages (package table)
- History (history table)
- Logs (logs table)
- Notifications (notifications table)

The data flows between processes, data stores, and external entities follow the standard DFD notation. The Authentication process receives users' credentials and ensures that they match those stored in the Users and Groups data store. The Real-Time Tracking process receives drivers' location GPS data, stores it in the Locations data store, and shares it with the students.

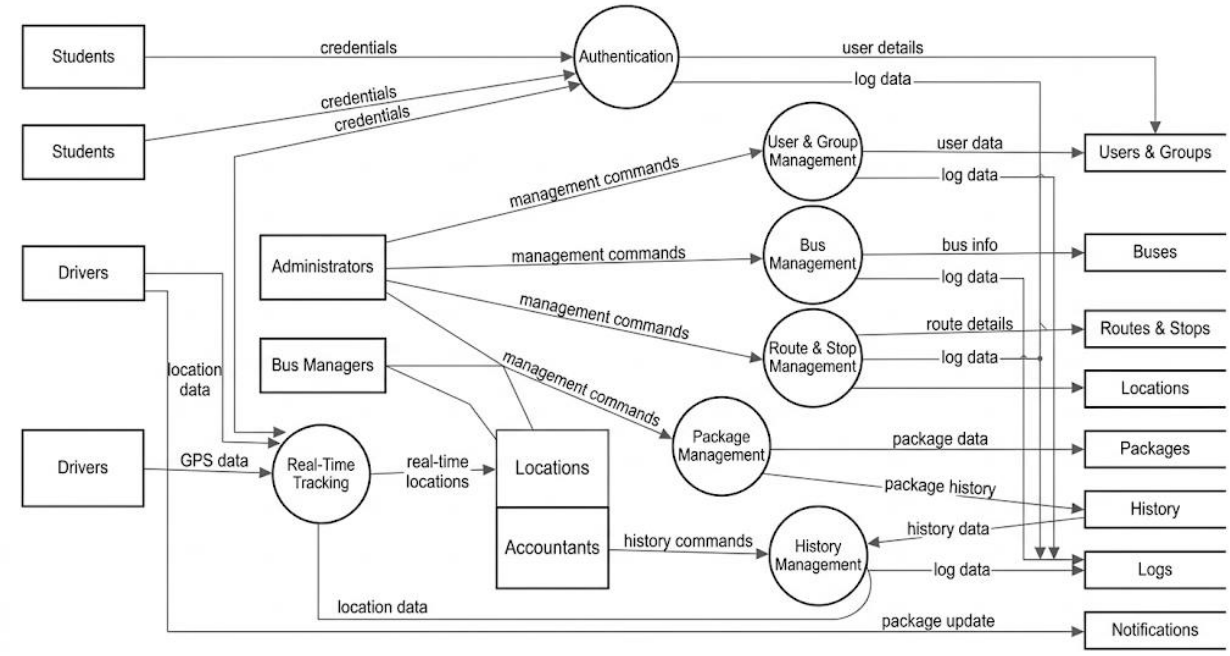


Figure 4.18: Level 1 Data Flow Diagram

4.12 API Architecture

The OBMMS API architecture is implemented as FastAPI for a monolithic backend. The backend API provides REST API endpoints and a WebSocket endpoint that are used for different purposes. The REST APIs support authentication, CRUD operations, package management, history, tracking data retrieval, and administration while the WebSocket endpoint is used for real-time GPS location exchange between drivers and students. All the REST endpoints are organized using separate routers with different prefixes and tags. The authentication is handled using JWT tokens. By logging in, the user gets a specific JWT token that should be included in the request headers. The backend checks whether the requested resource is allowed to the user's group and if so, allow access.

The main API routers in the project are:

Router	Prefix	Purpose
Authentication	/api/auth	Login, logout, permissions, groups, drivers, password change
Administration	/api/admin	Admin CRUD operations for users, groups, buses, stops, routes, packages, logs, history, location
Accounts	/api/accounts	Packages, history, models

Tracking	/api/tracking	Buses, active buses, my buses, stops, location, routes
Root	/	Root page, catch-all frontend path, WebSocket /ws

Table 4.1: API Routers in Project.

Method	Endpoint	Router	Description
POST	/api/auth/login/	Auth	User login and JWT generation
GET	/api/auth/login/check/	Auth	Check login status
GET	/api/auth/logout/	Auth	User logout
GET	/api/auth/permissions/	Auth	Get permissions of current user
POST	/api/auth/change_password/	Auth	Change user password
GET	/api/auth/groups/	Auth	Get all groups
GET	/api/auth/group/	Auth	Get current user group
GET	/api/auth/drivers/	Auth	Get drivers
GET	/api/admin/logs/all/	Admin	Get all logs
GET	/api/admin/users/all/	Admin	Get all users
GET	/api/admin/group/all/	Admin	Get all groups
GET	/api/admin/buses/all/	Admin	Get all buses
GET	/api/admin/stops/all/	Admin	Get all stops
GET	/api/admin/routes/all/	Admin	Get all routes
GET	/api/admin/packages/all/	Admin	Get all packages
GET	/api/admin/history/all/	Admin	Get all history records
GET	/api/admin/location/all/	Admin	Get all locations
POST	/api/admin/users/add/	Admin	Add new user
POST	/api/admin/group/add/	Admin	Add new group
POST	/api/admin/buses/add/	Admin	Add new bus
POST	/api/admin/stops/add/	Admin	Add new stop
POST	/api/admin/routes/add/	Admin	Add new route
POST	/api/admin/packages/add/	Admin	Add new package

POST	/api/admin/users/update/	Admin	Update user
POST	/api/admin/group/update/	Admin	Update group
POST	/api/admin/buses/update/	Admin	Update bus
POST	/api/admin/stops/update/	Admin	Update stop
POST	/api/admin/routes/update/	Admin	Update route
POST	/api/admin/packages/update/	Admin	Update package
POST	/api/admin/logs/delete/	Admin	Delete log
POST	/api/admin/users/delete/	Admin	Delete user
POST	/api/admin/group/delete/	Admin	Delete group
POST	/api/admin/buses/delete/	Admin	Delete bus
POST	/api/admin/stops/delete/	Admin	Delete stop
POST	/api/admin/packages/delete/	Admin	Delete package
POST	/api/admin/history/delete/	Admin	Delete history
GET	/api/accounts/packages/	Accounts	Get packages
GET	/api/accounts/history/	Accounts	Get history
GET	/api/accounts/models/	Accounts	Get available models
GET	/api/tracking/buses/	Tracking	Get buses
GET	/api/tracking/buses/active/	Tracking	Get active buses
GET	/api/tracking/buses/my/	Tracking	Get my buses
GET	/api/tracking/stops/	Tracking	Get stops
GET	/api/tracking/location/	Tracking	Get location data
GET	/api/tracking/routes/	Tracking	Get routes
GET	/	Root	Root endpoint
GET	/{full_path:path}	Root	Catch-all path for frontend
WebSocket	/ws	WebSocket	Real-time GPS location

Table 4.2: The detailed REST API endpoints

Although some routes are grouped under /api/admin, actual access is still controlled by the user's group permissions. A user can only perform an operation if the assigned group has the required permission level: **Not Allowed**, **Read Only**, or **Read and Write**. This API architecture provides a clean separation between authentication, administration, accounts, and tracking operations. The use of JWT-based authentication and group-based permissions ensures that every request is verified before access is granted. The WebSocket endpoint provides the real-time communication required for live bus tracking, while the REST endpoints support all standard CRUD and data retrieval operations.

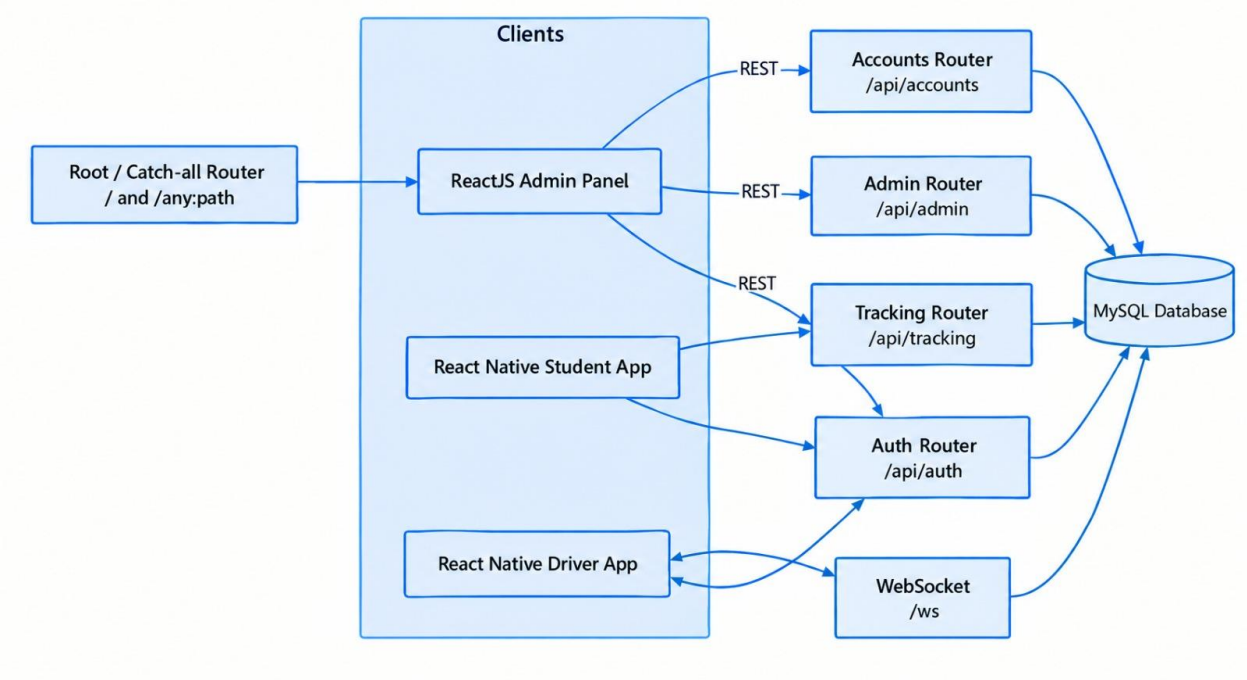


Figure 4.19: API Architecture of OBMMS

Chapter 5
Implementation

Chapter 5: Implementation

5.1 Development Environment

The Online Bus Management and Monitoring System (OBMMS) was implemented using a combination of web, backend, database, and mobile development technologies. The development process was carried out using **Visual Studio Code (VS Code)** as the primary development environment. The frontend of the system was developed using **React with Vite**, while the backend was implemented using **FastAPI and Python**. **MySQL** was used as the relational database management system, with **SQLAlchemy** used for database interaction and object-relational mapping. The mobile application was developed using **React Native and TypeScript**. Android Studio was used for Android-related development and testing. npm was used for managing JavaScript and React Native dependencies, while Python was used for backend development and dependency management. **XAMPP** was used as part of the local development environment, particularly for supporting the MySQL database environment. **PHPMyAdmin** was used for database management and inspection. Git and GitHub were used for version control and project source-code management. The completed system was deployed on an **Ubuntu-based Contabo VPS**.

The main technologies and tools used in the implementation are summarized below:

Component	**Technology/Tool**
Development Environment	Visual Studio Code
Web Frontend	React
Frontend Build Tool	Vite
Backend	FastAPI
Programming Language	Python
Database	MySQL
ORM	SQLAlchemy
Mobile Application	React Native
Mobile Programming Language	TypeScript
Android Development	Android Studio
Package Management	npm
Local Server Environment	XAMPP
Database Management	PHPMyAdmin
Real-Time Communication	WebSocket
Authentication	JWT
Password Hashing	Passlib
Version Control	Git and GitHub
Deployment	Contabo Ubuntu VPS

5.2 Frontend Implementation

The web frontend of OBMMS was implemented using **React** and **Vite**. React was used to develop the user interface using reusable components, while Vite was used as the frontend development and build tool. The frontend provides the administrative and management interface through which authorized users can access different system modules according to their assigned permissions.

The main frontend modules implemented in the system are:

5.2.1 Login

The Login module provides the authentication interface. Users enter their username and password to access the system. After successful authentication, the backend verifies the credentials and provides the required JWT-based authentication information.

5.2.2 Dashboard

The Dashboard provides an overview of the system and acts as the main entry point after authentication. The available information and operations depend on the permissions assigned to the user's group.

5.2.3 Users

The Users module is used for managing system users. Authorized users can perform user-related operations according to their assigned Read or Write permissions.

5.2.4 Groups

The Groups module manages user groups and their permissions. Groups are configurable, allowing different users to be assigned different levels of access to system resources.

5.2.5 Buses

The Buses module provides functionality for managing bus information. Authorized users can perform operations according to their assigned permissions.

5.2.6 Stops

The Stops module is used to manage bus stops. Stop information includes the stop name, description, geographic coordinates, campus status, and active status.

5.2.7 Routes

The Routes module manages route information. A route connects relevant stops and is associated with a bus and its assigned driver.

5.2.8 Packages

The Packages module is used to manage user transportation packages. Authorized users, particularly accountants with the required permissions, can create and manage packages and their active status.

5.2.9 History

The History module stores package-related transaction information. Access to history is controlled through permissions, while deletion of history is restricted to the administrator.

5.2.10 Locations

The Locations module offers access to bus location details. The location feature is linked to real-time bus tracking instead of being implemented as an independent CRUD resource. Frontend communicates with the FastAPI backend via the implemented API interfaces. The frontend does not directly interact with the database.

5.3 Backend Implementation

The backend of OBMMS was developed in **FastAPI and Python**. FastAPI provides the REST API layer through which the frontend and mobile application communicate with the backend. The backend is responsible for authentication, authorization, business logic, database operations, package management, and real-time communication. **SQLAlchemy** is used as the Object-Relational Mapping (ORM) layer between the FastAPI application and the MySQL database. This allows database entities to be represented through application models and provides a structured method for performing database operations.

The backend provides REST API functionality for the main CRUD resources, including:

- Users
- Groups
- Buses
- Stops
- Routes
- Packages
- History

Real-time location communication is handled separately using **WebSocket connections**. REST APIs are primarily used for normal data management operations, while WebSockets are used for continuous location communication. The backend also implements permission checking before allowing users to perform protected operations. Therefore, access to a resource depends on the permissions associated with the user's group.

5.4 Database Implementation

MySQL was used as the primary relational database management system for OBMMS. SQLAlchemy was used to establish communication between the FastAPI backend and the MySQL database. The database stores the main entities that are necessary to implement user management, access control, transportation management, package management, and transaction history.

The primary entities that have been implemented are:

- User
- Group
- Permission
- Bus
- Stop
- Route
- Package
- History
- Location

The major relationships implemented in the system include:

- **User → Group**
- **Group → Permission**
- **Route → Stops**
- **Route → Bus**
- **Bus → Driver**
- **Package → User**
- **History → Package**
- **History → User**

A route is associated with the relevant stops and bus. The bus is connected with its driver. One driver may drive more than one bus. The Package is related to the user, while the history table stores the data about the user's transactions, such as the Package, amount, date, and time. The user's Package has different attributes, such as Price, Month, Year, and Active status. Moreover, a user can have several packages. The database is accessed via SQL Alchemy, not through the front-end application or mobile app directly accessing the MySQL database.

5.5 Mobile Application Implementation

The mobile application of OBMMS was implemented using **React Native and TypeScript**. It acts as a client application for students and drivers and communicates with the backend services.

The mobile application provides the following major functionality:

- User login
- Navigation between application screens
- User profile
- Bus selection
- GPS location sharing for drivers
- Live bus tracking for authorized students
- Map display
- Stops and route information
- Package information
- Logout

The mobile application communicates with the backend to retrieve required information and to participate in the real-time bus tracking functionality. For drivers, the mobile application provides the functionality required to select a bus and share its current GPS location. For students, the mobile application provides access to available bus and route information and displays live bus locations on the map when the user is authorized to access the tracking information.

5.6 Authentication and Authorization

OBMMS implements authentication using a **username and password** mechanism. Email-based authentication is not used. After the user provides valid credentials, the backend verifies the credentials and generates a **JSON Web Token (JWT)**. JWT is used to authenticate subsequent requests to protected backend resources. Passwords are protected using **Passlib** password hashing functionality rather than storing passwords directly in plain text. The project uses a configurable token expiry mechanism. The default token expiry period is **30 days**. Token verification and refresh functionality is used to maintain authenticated sessions. Authorization is implemented using configurable groups and permissions. Instead of relying only on fixed application roles, users are associated with groups, and groups are assigned permissions.

The permission structure follows the following concept:

Group → Permission → Resource → Read / Write

The resources controlled by the permission system include areas such as:

- Users
- Buses
- Routes
- Locations
- Packages
- History

A user can perform an operation only when the user's group has the required permission. This provides controlled access to system functionality. For example, a user with Read permission can access the relevant information, while a user with Write permission can perform the permitted management operations. History deletion is restricted to the administrator.

5.7 Real-Time Tracking Implementation

The tracking of real-time bus location is one of the main features of OBMMS. The feature provides a way to obtain the current position of a bus from the driver's Android device and provide it to authorized students. The tracking architecture involves GPS and WebSocket communication.

The overall tracking process is as follows:

Driver Android Device → GPS → WebSocket → FastAPI Backend → WebSocket → Student Mobile Application → Map

The driver logs in to the mobile application and selects the bus that is being driven. The mobile device obtains its GPS coordinates and sends the location information to the backend through a WebSocket connection. The FastAPI backend receives the location data and broadcasts it to the appropriate connected clients through a designated WebSocket room. The location is then displayed on the student's mobile application map.

5.7.1 GPS Location Acquisition

GPS Location Getting Starts

When the Driver Chooses a Bus and Shares the Location

The application uses the driver's Android device as the tracking device. This implies that an ordinary Android phone or an android panel mounted in the bus can turn into a tracking device. The GPS supplies the current geographical coordinates of the bus.

The information comprises of:

- Bus ID
- Latitude
- Longitude

The driver has a Start/Stop Sharing control. Once the GPS location sharing is started, the application starts supplying the bus's current location to the backend. When the location sharing is stopped, the provision of the location is halted.

5.7.2 WebSocket Connection

The Web-Sockets are used to provide a constant two-way communication channel between the mobile applications and the FastAPI backend.

The implementation of the communication flow is as follows:

Driver Mobile Application → WebSocket → FastAPI Backend → WebSocket → Student Mobile Application

The WebSocket connection from the driver application is used to send location information. The student mobile applications create a WebSocket connection for receiving real-time location information. The backend is used to maintain the WebSocket connection for real-time communication. Unlike REST API, which is used for single request and single response, Web-Sockets provide a mechanism for maintaining real-time communication channels. Using Web-Sockets eliminates the need to make a normal HTTP request for each new piece of information.

5.7.3 Location Transmission

Once the GPS coordinates are obtained, the driver's app sends the necessary information to the WebSocket server.

The information sent includes:

- **bus_id**
- **latitude**
- **longitude**

The bus ID identifies the bus associated with the location, while latitude and longitude represent the current geographic position The FastAPI backend receives these location messages and processes them for real-time broadcasting.

5.7.4 Real-Time Broadcasting

Once the backend receives an updated location, it will broadcast the location information to the relevant allowed users. An allowed room is used for location broadcasting. Only the users who are connected to the relevant room can receive the location information. In this way, the backend can notify all eligible students about the bus location without sending the same location data individually via separate REST APIs. The student's mobile application gets the location information via Web-Sockets and updates the bus marker's position on the map. Hence, the real-time location data transfer flow can be represented as:

GPS → Driver WebSocket → FastAPI Backend →allowed Room → Student WebSocket → Mobile Map

5.7.5 Disconnection Handling

The tracking system takes care of disconnecting Web-Sockets and logging out the driver. By doing this, the driver is no longer sharing their location once they log out or if the WebSocket happens to be disconnected. The corresponding WebSocket connection gets removed, and the backend stops getting any location updates over that connection. By doing this, the driver's location is no longer shared with anyone after their tracking session connection is closed. This

mechanism ensures that real-time tracking is dependent on an active driver session and an active WebSocket connection.

5.8 Package Management Implementation

Package management is implemented as part of the transportation management functionality of OBMMS.

A package belongs to a specific user and contains information including:

- Package price
- Month
- Year
- Active status

An accountant with the required **Package Read and Write** permission can create and manage packages. Authorized users can also activate or deactivate packages. Package status is important for controlling access to transportation-related services. The system can use the active status of a package when determining whether the relevant user has the required package access. Package-related transactions are stored in the History entity. The history records information such as the package, user, amount, date, and time. This provides a record of package-related transactions and supports administrative monitoring of package activities.

5.9 Maps Integration

The mapping functionality of OBMMS is implemented using **OpenStreetMap** rather than Google Maps. For the web application, **React-Leaflet** is used to integrate OpenStreetMap-based maps into the React frontend. For the mobile application, the map functionality is provided through a **WebView**. This allows the mobile application to display the required map-based location information without using Google Maps. The map integration is particularly important for real-time bus tracking. When a student receives a new bus location through the WebSocket connection, the mobile application can display the corresponding bus position on the map.

The mapping architecture can thus be summarized as:

OpenStreetMap → React-Leaflet (Web) / WebView (Mobile)

This way, the mapping functionality is realized without having to rely on Google Maps services.

5.10 Security Implementation

Throughout the implementation of OBMMS, the security was taken under consideration. There are several mechanisms involved to secure the user accounts, API resources, and system data.

5.10.1 JWT Authentication

JWT is used to authorize the users after a successful log in. The protected API endpoints require the user's authentication token that permits access.

5.10.2 Password Hashing

User passwords in the application are not stored as plain-text values. The password hashing is implemented using the **Passlib** library in order to ensure the protection of the stored user credentials.

5.10.3 Group-Based Authorization

The system uses configurable groups and permissions for the purpose of controlling access to resources. This prevents users from accessing operations for which their group does not have the required permission.

5.10.4 Read and Write Permissions

Resources can be secured by Read and Write permissions. In this case, the system will be able to differentiate between users who have access only to information and users who have the ability to change information.

5.10.5 Protected APIs

Backend API endpoints are protected by authentication and authorization mechanisms. Any request to a restricted resource requires authentication and sufficient permissions for processing.

5.10.6 CORS

Cross-Origin Resource Sharing (CORS) is configure on the back-end of the project, Fast API to control communications between the back and the authorized front ends of the application.

5.10.7 Token Expiry

JWT tokens can have an expire period that is set up by an administrator. By default, the token expiry system uses 30-day duration. The process of tokens verification and refresh is applied to keep a session open.

5.10.8 Application Settings

The project uses setting.py for application configuration rather than an .env file. Configuration and security-related settings, including the secret key, are managed through the project's settings implementation.

5.10.9 Input Validation

Input validation is done on the incoming data before it is processed by the backend. This ensures that submitted values adhere to the expected data structures and requirements.

5.10.10 SQLAlchemy Database Protection

SQLAlchemy is applied to perform database operations between FastAPI backend and MySQL. The database operations are executed through an ORM layer, not by directly building queries from untrusted user input. This approach provides a systematic way of database access and prevents common SQL vulnerabilities.

5.10.11 HTTPS

The deployed system uses HTTPS to provide encrypted communication between clients and the deployed application services. This protects data transmitted between users' devices and the server during system operation.

5.11 Version Control

Git and GitHub were used for source-code version control throughout the development of OBMMS. The project source code was maintained in a GitHub repository, allowing changes to be tracked through commits and providing a centralized location for project development. Git was used to record development changes and maintain different versions of the project source code. GitHub was used as the remote repository for storing and managing the project. Version control also provides a mechanism for maintaining the project history and recovering previous versions when required.

5.12 Deployment

The completed OBMMS system was deployed on a **Contabo Ubuntu VPS**. The Ubuntu-based VPS provides the server environment required to host the backend services and make the system accessible to its client applications. The deployed architecture allows the web frontend, backend services, database services, and real-time WebSocket communication to operate as part of the overall system. The deployment environment supports the FastAPI backend and its communication with the MySQL database. The WebSocket functionality is also available in the deployed environment so that driver devices can transmit real-time location information and authorized student applications can receive it. The deployment of the system on a VPS moves the application from the local development environment to a remotely accessible server environment, allowing the implemented web and mobile clients to communicate with the backend during actual system operation.

Chapter 6
Testing and Evaluation

Chapter 6: Testing and Evaluation

6.1 Testing Strategy

The testing strategy for the Online Bus Monitoring and Management System (OBMMS) was based on manual testing. None of the automated testing frameworks or tools such as Postman, Swagger UI, Pytest, or Jest were employed. The backend and frontend were developed concurrently while testing took place in parallel with the development process. This enabled immediate verification of the API responses and the behavior of user interfaces. The web administration panel was tested manually using a browser. The mobile application was tested on both Android emulators and physical devices. The testing took a modular approach where each module was tested independently first before testing all of them conjoined to test the integrated system. Permission levels were tested extensively by logging in as different users and groups. The three permission levels — Not Allowed, Read Only, and Read and Write — were verified for each resource. The administrator site was used repeatedly to confirm that CRUD operations and permission restrictions behave as expected.

6.2 Functional Testing

Functional testing was applied to all the major modules of the system, including Users, Groups, Buses, Stops, Routes, Packages, History, Location, and Logs. The Notifications module has been left unimplemented, and each of the described modules has been subject to testing individually to ensure the proper functionality of each segment. The modules have also been tested collectively to assess the interaction between them.

The following functions have been tested:

- User login, logout, and change password
- User and group management with different permission levels
- Bus, stop, and route CRUD operations
- Package creation, activation, and deactivation
- History viewing and deletion (admin only)
- Location sharing and real-time tracking
- Permission enforcement for Read Only and Read and Write access

The testing also included the use of invalid input data and the examination of edge cases. In addition, some tests were carried out using the admin panel to ensure that unauthorized attempts to make changes would be rejected.

6.3 API Testing

API Testing was performed manually, without the use of Postman or Swagger UI, by sending requests from frontend applications and reviewing responses from FastAPI backend. Both authenticated and unauthenticated requests were sent and reviewed.

The following behaviors of the API were checked:

- Requests without a valid JWT token returned HTTP 401 (Unauthorized).
- Requests with a valid token but insufficient permissions returned HTTP 403 (Forbidden).
- Response formats were checked and consumed correctly by the frontend.
- All endpoints were reviewed carefully to ensure proper request handling.

The Authorization header was used to send a valid JWT, which the backend recognized and allowed to proceed based on the group that issued it.

6.4 Integration Testing

Integration Testing of the application involved testing between the frontend, backend, and database. ReactJS web panel and React Native mobile application were integrated with the FastAPI backend to form a unit and were tested together. MySQL database was tested on a continuous basis during development. One of the significant integration tests was the testing of the WebSocket server, backend, and mobile application. The test was done in a moving vehicle to observe the location tracking in real-time. The driver's mobile application sent the GPS data using WebSocket to the backend, and the student's application was receiving and displaying the location on the map. The integration was successful, although there were some issues with location sharing and map display. Integration Testing of the application involved testing between the frontend, backend, and database. ReactJS web panel and React Native mobile application were integrated with the FastAPI backend to form a unit and were tested together. MySQL database was tested on a continuous basis during development. One of the significant integration tests was the testing of the WebSocket server, backend, and mobile application. The test was done in a moving vehicle to observe the location tracking in real-time. The driver's mobile application sent the GPS data using WebSocket to the backend, and the student's application was receiving and displaying the location on the map. The integration was successful, although there were some issues with location sharing and map display.

6.5 Mobile Application Testing

The application has two target users: students and drivers. The first one has the option to choose buses upon the driver's login into the application. The same application is used for students to mark buses and their location. The application was tested on Android emulators and devices. The minimum version of Android for proper functioning is 11. Android 11 and higher were able to run the application.

Some test results can be seen below:

- GPS permission was relatively easy to obtain.
- Issues with location: huge challenge to implement correctly
- Map: nested scrolling issue required several iterations to fix
- Offline: switching to polling when disconnected and back to polling when connected

- worked as expected
- The application was tested on one laptop and two Android devices simultaneously, and
- the system remained stable

6.6 Real-Time Communication Testing

Real-time communication was evaluated using the WebSocket endpoint. The WebSocket connection appeared stable during testing. Multiple clients were connected at the same time (one lap-top and two Android devices) without any loss of connection. The following real-time scenarios were tested:

- Driver location sharing begins when the driver selects a bus.
- Location updates are broadcast to connected students.
- When the driver disconnects or logs out, location sharing is stopped.
- Latency was not formally measured, but no noticeable delay was observed during the live test.

Load testing was not performed because the system was deployed on a minimal VPS package. The number of concurrent connections was limited to the available test devices.

6.7 Performance Testing

The system was not subjected to formal performance testing. It was installed on a minimal VPS package, precluding high-load simulations. The response times were not measured, and database queries were not benchmarked. Nevertheless, during routine operations, the system exhibited satisfactory responsiveness. The real-time tracking component performed satisfactorily with few concurrent users. Further performance tests are advisable to establish the system's capacity to handle increased loads.

6.8 Security Testing

The security testing involved testing the application's authentication and authorization mechanisms. This includes the following:

- The test endpoints required a JWT token to be sent in the request headers. Without the token, the application returned a 401 error.
- Users were blocked when they attempted to access resources that required higher permissions; thus, the application returned a 403 error.
- The passwords were stored in the database as hash values using PHPMyAdmin. This was tested by attempting to log in with several test accounts.
- The application was able to prevent unauthorized access to the system. This was tested by sending requests with invalid tokens.

I did not test the JWT expiry time. The bypassing of permissions was not explicitly tested; however, all endpoints were checked for proper permissions, and the application employed

several checks to ensure security. I did not test SQL injection; however, the application uses an ORM to interact with databases, thus preventing SQL injection attacks.

6.9 Usability Evaluation

The usability of the system was informally evaluated. A formal questionnaire was not developed and distributed. Instead a demonstration of the system was provided to teachers and colleagues and their feedback was gathered and reviewed. Several usability issues were identified, mostly related to the user experience and interface design, which will be addressed in the future work. Test cases were not formally documented. The testing process was mainly exploratory and manual. It could be beneficial to perform a more thorough and structured usability assessment in the future.

6.10 Testing Results

Most of the described modules passed the manual testing procedure and performed their functions correctly. However, several issues were discovered:

- **Offline login:** if a user attempts to log in when they are offline, an extended error log appears, and the request handling is not done gracefully.
- **History:** history is not available in the mobile application.
- **Permission errors:** if a student does not have Read permission for stops or buses, an error occurs.
- **Private notifications:** there were some issues with private notifications.
- **Push notifications**: push notifications have some unresolved issues.
- **Offline:** there are several unresolved issues with the offline mode.

Some of the discovered bugs were fixed during the testing procedure, while others will be resolved in the future. The system under test is functioning but needs improvements.

6.11 System Limitations

The existing software of the OBMMS has some shortcomings that need some improvement:

- there is no parent portal implemented, however, the creation of a parent group in case of it is needed is possible.
- the push notification is not properly implemented.
- the offline mode has some issues.
- the application requires an Internet connection for logging in and tracking.
- the performance and scalability of the software were not examined.
- the question of tokens' validity and their refreshing mechanism were not studied.
- there is no test automation framework.
- GPS has a good accuracy, but the implementation of the sharing of the location and the display on the map needs much development.
- there are some issues with the application's usability which are connected to the UX.

Chapter 7
Conclusion & Future Enhancements

Chapter 7: Conclusion & Future Enhancements

7.1 Project Summary

The Online Bus Monitoring and Management System (OBMMS) was conceived to resolve two identified problems among university students and administration, namely, lack of transportation information about the buses and manual fee management. The system provides a mobile application for students and drivers, a web administration panel for management staff, and a FastAPI monolithic backend with a MySQL database. It facilitates real-time bus tracking via WebSocket communication. Drivers log in to the mobile application, choose a bus they will drive, and send their GPS location data. It is then relayed to the FastAPI backend and broadcasted to the students who have access to it and can see the bus location on a map. The system provides a package-based access mechanism. Students who do not have location access can use an existing package to access bus tracking.

A permission-based access control was implemented using groups. Users are put into groups and a set of permissions is assigned to each group for every resource in the system. The permission levels are Not Allowed, Read Only, and Read and Write. Through this, the university management can define custom user roles according to their needs. The backend provides REST API endpoints for authentication, administration, accounts, and tracking plus a WebSocket endpoint for real-time communication. The authentication mechanism uses the JWT standard to secure all protected endpoints. The system also provides logs and history records for auditing and accountability purposes.

7.2 Achievement of Objectives

The project objectives were to provide real time tracking of buses, reduce the waiting time, ensure students' safety, and develop a secured online fee management system. Below are the discussions on to what extent each objective was met.

Objective I: To provide real-time tracking of university buses

This objective has been achieved. The system allows to track the university buses in real time using the WebSocket connection. The drivers send their coordinates from the mobile application, while the students see the current location of the buses on the map. This feature was tested on a moving vehicle and had a stable connection.

Objective II: To reduce waiting time and prevent students from missing buses

The objective was achieved to some extent. Students can monitor the location of buses and prepare to leave for school. Push notifications and eta tracking would help students know if the bus is near and when they should expect it to arrive. These were not integrated into the application, which means that they have to constantly check. This means that they can't see the buses' location in real-time.

Objective III: To improve the safety and monitoring of students during travel

The objective was partially achieved. It is possible to monitor bus locations in real-time. Administrators can access the location of every bus via the admin panel of the system. However, the features of geofencing, route deviation detection, and emergency vehicle tracking alerts were not implemented in the system. The program is not available for parents; nevertheless, a parent group can be created in case of necessity.

7.3 System Benefits

The OBMMS provides several benefits to students, drivers, and university administration:

For Students:

- Students can view the live location of buses and plan their travel accordingly.
- The mobile application provides access to bus, route, and stop information.
- Students with active packages can access bus tracking without requiring direct location permissions.

For Drivers:

- Drivers can easily select an assigned bus and share its GPS location.
- A driver can be assigned to multiple buses and can select the appropriate bus before starting location sharing.

For University Administration:

- The admin panel offers complete CRUD operations for users, groups, buses, stops, routes,
- packages, and history.
- The permission-based group system provides flexible access control that is not reliant on hard-coded roles.
- Logs and history records ensure accountability and auditability.
- The system reduces the need for manual management of transportation and packages.

For the University:

- The system improves transparency in bus operations.
- It provides a foundation for digital transformation of transportation management.
- It can be extended to include parents and additional management roles in the future.

7.4 Limitations

The current version of OBMMS has several limitations:

- The push notifications are not fully functioning.
- The estimated time of arrival (ETA) calculation feature is not implemented.
- The geofencing and route deviation detection capabilities are not available.
- Parent access to the system is not enabled, though a parent group can be created.
- The online payment gateway feature is not implemented.

- The offline mode is not working correctly; attempting to log in when there is no internet connection results in a long error log and not properly handled requests.
- History is not shown in the mobile application.
- If a student does not have Read permissions for stops or buses, an error occurs.
- The private notifications have some unresolved issues.
- The JWT expiry and token refresh were not tested.
- The performance and scalability were not tested.
- There is no automated test framework; all tests were done manually.
- Some UX-related usability issues were discovered.

These issues give a good understanding of what should be done in the future.

7.5 Lessons Learned

The development of OBMMS provided several valuable lessons:

- **Real-time communication is challenging:** Using Web-Sockets for real-time communication isn't easy. You have to think about connections and disconnections and what to do if the connection is lost. Testing it all in a car made it easier to see what would actually happen.
- **Map integration on mobile is complex:** Creating an integrated map system is not a trivial task. Displaying the current location on the map involved nested scrolling which was initially tricky to implement and had to be tested several times before it worked correctly. This taught me to always test mobile-specific interactions.
- **Permission-based access control requires careful planning:** Permission-based access control requires much planning. Creating a group-based permission system that was flexible enough was a must if different management roles were going to be satisfied. Having each permission tested by hand verified that they worked correctly.
- **Manual testing has limitations:** Manual testing is limited in scope. Without automated testing, regression testing become challenging. Formal test cases and automated testing frameworks will increase the reliability in further developments.
- **Offline behavior must be designed early:** Offline behavior should be designed in early phases. Handling network disconnection is important for mobile applications. Current implementation uses polling as a fallback when the WebSocket disconnects. Login procedure and error handling when being offline needs to be improved.
- **User experience matters:** User experience is something that should not be ignored. Unofficial feedback from teachers and peers has revealed some UX problems that I hadn't noticed while developing the app. Performing a formal usability evaluation would be good to address these issues.

7.6 Future Enhancements

Future developments of the Online Bus Monitoring and Management System (OBMMS) should aim at enabling fully functional push notifications to notify students when a bus nears their selected stop, including ETA feature that estimates and shows the time of arrival based on current vehicle position and route. The application can also be augmented with geo-fencing feature to track deviations from a route and trigger appropriate administrator alerts. The parent portal can be incorporated either in the form of the parent group with specific access permissions or a separate portal designed for the parents.

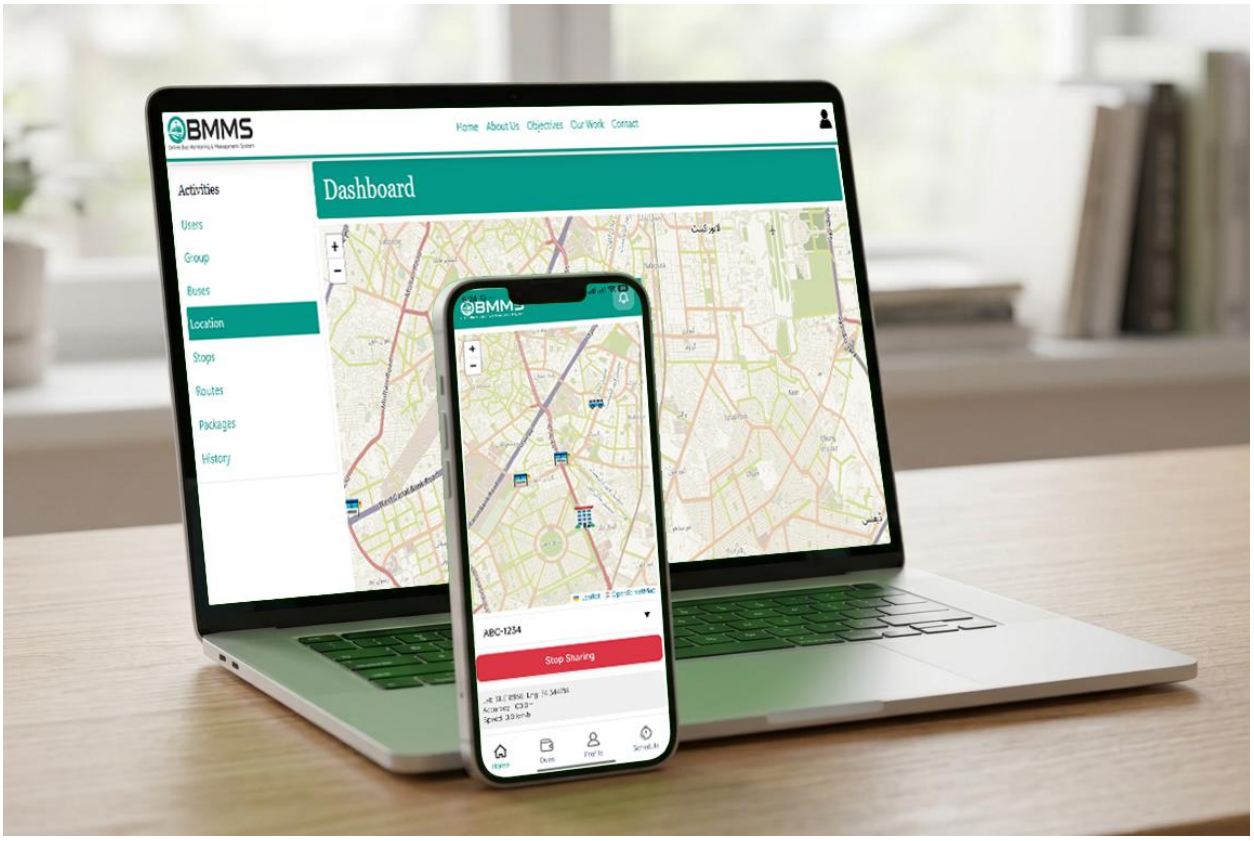
Also, an online payment system can be introduced to enable the system to support subscription fees for packages. The offline mode can be improved by tweaking the application to handle login, errors, and requests in the absence of internet connections. In addition, the mobile application can be adjusted to view past package and transactions details. Other alterations include addressing permission errors for students that try to access information they are not allowed to see, such as where the “Read” permission for stops or buses has not been granted. Also, the mobile and web application can be configured to support JWT expiry and refresh tokens to enhance security and reliability. Performance and scaling tests can be conducted to determine the performance of the system under extreme loads. Testing tools can also be utilized to test the backend, frontend, and mobile application test automation to ensure quality-enhanced products with fewer bugs.

A formal usability test can be conducted to identify other teething issues and address them. It is also important to consider the application’s compatibility with older versions of android if necessary. Finally, the system should be improved to allow universities to define roles, groups, and permissions without changing the system’s code for better flexibility and scalability in the future. The proposed changes will enhance the reliability, versatility, and security of OBMMS to support a wide array of features for managing university transportation and fee systems.

References

1. https://vite.dev/guide/
2. https://www.w3schools.com/react/
3. https://reactrouter.com/home
4. https://tailwindcss.com/docs/installation/using-vite
5. https://www.geeksforgeeks.org/python/fastapi-introduction/
6. https://www.tutorialspoint.com/fastapi/fastapi_uvicorn.htm
7. https://www.tutorialspoint.com/sqlalchemy/
8. https://docs.sqlalchemy.org/en/20/orm/quickstart.html
9. https://planetscale.com/blog/using-mysql-with-sql-alchemy-hands-on-examples
10. https://www.tutorialspoint.com/sqlalchemy/
11. https://www.svgrepo.com/
12. https://www.tutorialspoint.com/fastapi/fastapi_static_files.htm
13. https://stackoverflow.com/questions/287871/how-do-i-print-colored-text-to-the-terminal#:~:text=termcolor.COLORS%20gives%20you%20a,other%20platforms%2C%20Colorama%20does%20nothing
14. https://pyjwt.readthedocs.io/en/stable/
15. https://react-leaflet.js.org
16. https://www.openstreetmap.org
17. https://www.w3schools.com/tags/ref_httpmessages.asp
18. https://medium.com/@farhanahmedindia/complete-guide-deploying-a-flask-app-on-apache-ubuntu-c2f5d7b17e20
19. https://reactnative.dev/docs/set-up-your-environment
20. https://reactnative.dev/docs/navigation
21. https://reactnative.dev/docs/network
22. https://www.npmjs.com/package/react-native-webview
23. https://www.npmjs.com/package/react-native-keychain
24. https://fastapi.tiangolo.com/advanced/websockets/
25. https://archive.reactnative.dev/docs/geolocation#:~:text=iOS,result%20in%20a%20hard%20crash.
26. https://github.com/moeezraza35/mr_wshandler_fastapi
27. https://github.com/moeezraza35/mr-wshandler-react

Appendix A: **Project Presentation**



github.com/moeezraza35/OBMMS

OBMMS Project Report

GRADEMARK REPORT

FINAL GRADE

/0

GENERAL COMMENTS

PAGE 1

PAGE 2

PAGE 3

PAGE 4

PAGE 5

PAGE 6

PAGE 7

PAGE 8

PAGE 9

PAGE 10

PAGE 11

PAGE 12

PAGE 13

PAGE 14

PAGE 15

PAGE 16

PAGE 17

PAGE 18

PAGE 19

PAGE 20

PAGE 21

PAGE 22

PAGE 23

PAGE 24

PAGE 25

PAGE 26

PAGE 27

PAGE 28

PAGE 29

PAGE 30

PAGE 31

PAGE 32

PAGE 33

PAGE 34

PAGE 35

PAGE 36

PAGE 37

PAGE 38

PAGE 39

PAGE 40

PAGE 41

PAGE 42

PAGE 43

PAGE 44

PAGE 45

PAGE 46

PAGE 47

PAGE 48

PAGE 49

PAGE 50

PAGE 51

PAGE 52

PAGE 53

PAGE 54

PAGE 55

PAGE 56

PAGE 57

PAGE 58

PAGE 59

PAGE 60

PAGE 61

PAGE 62

PAGE 63

PAGE 64

PAGE 65

PAGE 66

PAGE 67

PAGE 68

PAGE 69

PAGE 70

PAGE 71

PAGE 72

PAGE 73

PAGE 74

PAGE 75

PAGE 76

PAGE 77

PAGE 78

PAGE 79

PAGE 80

PAGE 81

PAGE 82

PAGE 83

PAGE 84

PAGE 85

PAGE 86

PAGE 87

PAGE 88

PAGE 89

PAGE 90

PAGE 91

PAGE 92

PAGE 93

PAGE 94

PAGE 95

PAGE 96